DAYANANDA SAGAR UNIVERSITY

Kudlu Gate, Hosur Road, Bengaluru – 560068

SCHOOL OF COMPUTER APPLICATIONS

A Project Report on

AI-Based Multi-Modal Secure Code Analysis Platform

Submitted in Partial fulfilment of the requirements for the award of the Degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by

[STUDENT NAME - REQUIRES VERIFICATION]

[USN NUMBER - REQUIRES VERIFICATION]

Under the guidance of

[GUIDE NAME AND DESIGNATION - REQUIRES VERIFICATION]

[SUBMISSION MONTH AND YEAR - REQUIRES VERIFICATION]

DAYANANDA SAGAR UNIVERSITY

Kudlu Gate, Hosur Road, Bengaluru – 560068

SCHOOL OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

This is to certify that the project titled ‘AI-Based Multi-Modal Secure Code Analysis Platform’ is a bonafide record of the work undertaken by [STUDENT NAME - REQUIRES VERIFICATION] ([USN NUMBER - REQUIRES VERIFICATION]) in partial fulfillment of the requirements for the award of the Master of Computer Applications degree at the School of Computer Applications, Dayananda Sagar University, Bengaluru, during the academic year 2025-2026.

GUIDE **DEAN**

Internal Examiner **External Examiner**

SCHOOL OF COMPUTER APPLICATIONS

DECLARATION

I, [STUDENT NAME - REQUIRES VERIFICATION], a student of the Master of Computer Applications program at the School of Computer Applications, Dayananda Sagar University, hereby declare that this project, titled 'AI-Based Multi-Modal Secure Code Analysis Platform', is the result of my independent work carried out under the supervision of [GUIDE NAME AND DESIGNATION - REQUIRES VERIFICATION].

I am submitting this project work in partial fulfillment of the requirements for the award of the Master of Computer Applications degree by Dayananda Sagar University, Bengaluru, for the Academic Year 2025-2026.

I further affirm that this project report, in whole or in part, has not been submitted for the award of any other degree or diploma at this or any other university/institution.

Signed by me on: [VIVA DATE - REQUIRES VERIFICATION]

Signature of the Candidate

This is to certify that the project work submitted by [STUDENT NAME - REQUIRES VERIFICATION] has been carried out under my guidance, and the declaration made by the candidate is true to the best of my knowledge.

Signature of the Guide

Date: [VIVA DATE - REQUIRES VERIFICATION]

ACKNOWLEDGEMENT

I express my heartfelt gratitude and salutations to my beloved and highly esteemed institute, Dayananda Sagar University, for providing me with an excellent platform to learn and grow. I am deeply thankful for the unwavering support and encouragement extended throughout my academic journey.

I extend my sincere gratitude to Dr. D. Hemachandra Sagar, Chancellor, and Dr. Premachandra Sagar, Pro Chancellor, Dayananda Sagar University, for their visionary leadership and continuous support in fostering an environment of academic excellence and innovation.

I am profoundly grateful to Dr. B.S. Satyanarayana, Vice Chancellor, and Dr. Puttamadappa C, Registrar, Dayananda Sagar University, for their invaluable guidance and commitment to ensuring a world-class education at the university.

I extend my heartfelt thanks to Dr. S. Senthil, Professor and Dean, School of Computer Applications, Dayananda Sagar University, for his valuable suggestions, expert advice, and constant support throughout my project.

I am deeply indebted to my guide, [GUIDE NAME AND DESIGNATION - REQUIRES VERIFICATION], School of Computer Applications, for the continuous support, necessary advice, and invaluable guidance that played a significant role in shaping the outcome of my project.

I would also like to extend my appreciation to all the teaching and non-teaching staff members of the School of Computer Applications for their support and assistance during my academic tenure.

Lastly, I am forever grateful to my parents and friends, whose unwavering support, motivation, and encouragement have been my strength throughout this journey.

[STUDENT NAME - REQUIRES VERIFICATION]

[USN NUMBER - REQUIRES VERIFICATION]

CONTENTS

Chapter 1: INTRODUCTION....1
1.1 PROJECT OVERVIEW....1
1.2 STATEMENT OF THE PROBLEM....2
1.3 BRIEF DESCRIPTION OF THE PROJECT....2
1.4 OBJECTIVES OF THE PROJECT....3
1.5 SCOPE OF THE PROJECT....3
1.6 SOFTWARE AND HARDWARE REQUIREMENTS....4
1.6.1 Software Specifications....4
1.6.2 Hardware Specifications....4
1.7 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS....5
1.8 COMPANY/DOMAIN PROFILE....6
Chapter 2: LITERATURE SURVEY....7
2.1 EXISTING RESEARCH AND RELATED WORK....7
2.2 GAPS IN EXISTING SYSTEMS....9
2.3 OVERVIEW OF TECHNOLOGIES USED....10
2.4 COMPARISON OF DIFFERENT APPROACHES....11
Chapter 3: METHODOLOGY & SYSTEM ANALYSIS....13
3.1 EXISTING SYSTEM....13
3.1.1 Overview of Current System....13
3.1.2 Challenges and Limitations....13
3.2 PROPOSED SYSTEM....14
3.2.1 System Architecture....14
3.2.2 Features and Functionalities....14
3.3 DATASETS USED....15
3.3.1 Dataset Description....15
3.3.2 Data Collection Process....15
3.3.3 Data Preprocessing Techniques....15
3.4 MACHINE LEARNING MODEL/ALGORITHM SELECTION....16
3.4.1 Algorithm Justification....16
3.4.2 Training and Testing Data Split....16
3.4.3 Feature Selection and Engineering....16

3.4.4 A discovered and corrected data-alignment defect 17
3.5 FEASIBILITY STUDY 18
3.5.1 Technical Feasibility 18
3.5.2 Economic Feasibility 18
3.5.3 Operational Feasibility 18
Chapter 4: SYSTEM DESIGN AND DEVELOPMENT 20
4.1 SYSTEM ARCHITECTURE (HIGH-LEVEL DESIGN) 20
4.1.1 Conceptual Design 20
4.1.2 Component Diagram 21
4.2 DETAILED DESIGN (LOW-LEVEL DESIGN) 21
4.2.1 Data Flow 21
4.2.2 Use Case Diagram (described) 22
4.2.3 Activity Diagram (Scan → Analysis → Remediation → Re-scan, described) 22
4.2.4 Database Design 22
4.3 INPUT/OUTPUT INTERFACE DESIGN 23
4.3.1 User Interface (UI) Screens 23
4.3.2 API Design 24
4.4 MODULE DESCRIPTION 25
4.4.1 Description of Each Module 25
4.4.2 Functionality and Interaction 25
4.5 CI/CD AND DEPLOYMENT WORKFLOW 25
Chapter 5: IMPLEMENTATION & CODING 27
5.1 PROGRAMMING LANGUAGE AND FRAMEWORK USED 27
5.2 ALGORITHMIC APPROACH 27
5.2.1 AST Feature Extraction (Pseudocode) 27
5.2.2 Flowchart (Feature Fusion → Prediction, described) 28
5.2.3 Model Training and Fine-Tuning 28
5.2.4 Code Snippets and Explanation 28
5.3.1 Key Functionalities 30
Chapter 6: SOFTWARE TESTING 31
6.1 TESTING STRATEGIES 31
6.1.1 Unit Testing 31
6.1.2 Integration Testing 31

6.1.3 System Testing....31
6.2 TEST CASES....32
6.2.1 Sample Test Cases with Expected vs. Actual Results....32
6.3 PERFORMANCE METRICS AND EVALUATION....33
Chapter 7: RESULTS AND DISCUSSION....35
7.1 EXPERIMENTAL SETUP....35
7.1.1 Hardware/Software Setup....35
7.1.2 Model Hyperparameters....35
7.2 MODEL PERFORMANCE ANALYSIS....35
7.2.1 Final Results....35
7.2.2 Training vs. Testing Performance / Bias-Variance Reading....36
7.2.3 Model Comparisons....37
7.3 COMPARISON WITH EXISTING SYSTEMS — AND A DISCLOSED CORRECTION....37
7.4 VISUALIZATION OF RESULTS....39
7.5 SCREENSHOTS OF WORKING SYSTEM....41
Chapter 8: CONCLUSION AND FUTURE ENHANCEMENTS....46
8.1 CONCLUSION....46
8.2 LIMITATIONS OF THE PROJECT....47
8.3 SCOPE FOR FUTURE ENHANCEMENTS....48
REFERENCES....49
RESEARCH PAPERS (VERIFIED VIA ORIGINAL SOURCE/ABSTRACT BEFORE INCLUSION — SEE REPORT/BLUEPRINT.MD)....49
DATASETS AND TOOLS....50
FRAMEWORKS AND APIS USED....50
PROJECT'S OWN INTERNAL DOCUMENTATION (PRIMARY SOURCES VERIFIED AND CITED THROUGHOUT THIS REPORT)....51

Chapter 1: INTRODUCTION

1.1 PROJECT OVERVIEW

Software vulnerabilities in C and C++ codebases remain a persistent source of security incidents, largely because the two dominant families of automated detection techniques each look at code from only one angle. Rule-based static analyzers match code against known dangerous patterns — unsafe functions, missing bounds checks, unchecked pointer arithmetic — and are precise on the patterns they know but blind to anything structurally different. Learning-based approaches built purely on token or sequence embeddings, on the other hand, capture semantic similarity to previously seen vulnerable code but have no explicit notion of control flow, nesting depth, or the structural shape of a function.

The **AI-Based Multi-Modal Secure Code Analysis Platform** is built on the premise that these two views are complementary rather than competing. The platform extracts three independent descriptions of every analyzed function — a semantic embedding produced by a pre-trained transformer model, a set of structural features derived from the function's Abstract Syntax Tree (AST), and a set of classical software-engineering metrics computed directly from the source text — concatenates them into a single feature vector, and classifies the fused representation with a gradient-boosted tree ensemble. The resulting model is exposed through a full web application rather than a notebook or command-line script: a FastAPI backend serves predictions, persists scan history, and performs auxiliary analyses (dependency graphs, call graphs, a CWE knowledge base, patch suggestion and re-scan), and a React/ TypeScript frontend presents these results across a set of purpose-built pages (a code scanner, a per-function analysis view, a recommendations view, an automated-patch view, several graph visualizations, and a GitHub repository scanning mode).

This report documents the complete pipeline as actually implemented and deployed — from the raw Devign dataset through preprocessing, three-branch feature extraction, feature fusion, model training, and the FastAPI/React application built on top of the trained model — and reports its measured performance honestly, including a data-fusion defect that was discovered during a later audit and corrected before the final models were trained (Section 6.4 and Chapter 7 discuss this in detail).

1.2 STATEMENT OF THE PROBLEM

Given a C/C++ function, determine whether it contains a security vulnerability, using a single model that combines semantic, structural, and metric-based signals, and deliver that determination through a usable application rather than only as an offline evaluation script. Single-modality baselines — either a fine-tuned transformer working on raw token sequences alone, or a graph neural network working on structural representations alone — each leave out information the other captures; the problem this project addresses is whether fusing all three signal types into one feature vector, classified by a conventional ensemble model, is a viable and practical alternative for function-level vulnerability triage, and what such a system looks like as a deployable tool rather than a research script.

1.3 BRIEF DESCRIPTION OF THE PROJECT

The project is organized around a linear offline pipeline and an online serving application built on top of its output:

- **Data preparation.** The Devign dataset (function-level C/C++ code labeled vulnerable/not-vulnerable) is cleaned — null values, duplicates, empty functions, and abnormally short functions are removed — leaving 27,231 usable samples.
- **Three-branch feature extraction.** Every retained function is independently converted into (a) a 768-dimensional CodeBERT [CLS] embedding, (b) an 11-dimensional AST feature vector obtained by parsing the function with tree-sitter's C grammar and traversing the resulting syntax tree, and (c) a 13-dimensional set of software metrics (line counts, branching/looping counts, nesting depth, operator and pointer usage, comment density) computed directly from the source text.
- **Feature fusion.** The three branches are joined on a stable per-function identifier into a single 792-dimensional feature vector per function.
- **Model training.** XGBoost and Random Forest classifiers are trained on an 80/20 stratified split (21,784 train / 5,447 test, random_state=42); XGBoost is selected as the primary model on the basis of its higher F1-score and AUC-ROC (Chapter 3.2.2 and Chapter 7 give the full comparison).
- **Application layer.** A FastAPI backend loads the trained model once at startup, reuses the same feature-extraction code for live requests, persists every scan to a

local SQLite store, and exposes it through a React frontend with dedicated pages for scanning, per-function analysis, remediation recommendations, automated patch application with re-scanning, dependency/call/knowledge-graph visualization of a whole uploaded project, and direct scanning of a public GitHub repository.

1.4 OBJECTIVES OF THE PROJECT

1. Build a reproducible offline pipeline that extracts semantic (CodeBERT), structural (AST), and metric-based features from C/C++ functions and fuses them into a single representation.
2. Train and compare tree-ensemble classifiers (XGBoost, Random Forest) on the fused representation and select the better-performing model using standard classification metrics (accuracy, precision, recall, F1-score, AUC-ROC, confusion matrix).
3. Package the trained pipeline behind a FastAPI service that can score both a single pasted function and a batch of uploaded files or an entire cloned GitHub repository.
4. Present the results through a React web interface that goes beyond a bare verdict — showing per-feature contribution, an embedding-space visualization, rule-based secure-rewrite suggestions, an automated patch-and-re-scan loop, and whole-project graph views (file dependencies, cross-file call graphs, and a unified knowledge graph linking files, functions, CWE entries, and predictions).
5. Verify every reported number against the project's own trained-model artifacts rather than carrying forward earlier, pre-training placeholder figures, and disclose the feature-fusion correction that was required to reach the final reported metrics.

1.5 SCOPE OF THE PROJECT

The system operates at **function-level granularity** on **C/C++ source code** drawn from the Devign dataset's distribution (real-world functions collected from open-source C/C++ projects). It classifies a function as vulnerable or not-vulnerable and estimates a probability/confidence for that verdict; it does not attempt to localize the vulnerability to a specific line with certainty (the line-level highlighting in the

application is a separate, deterministic pattern scanner used as a corroborating signal, not the ML model's own explanation) and does not cover languages other than C/C++. The GitHub-scanning mode operates on public repositories only, clones a bounded number of recent commits, and is explicit in its UI about which files were skipped and why, rather than silently presenting a partial scan as a complete one. The project targets a single-operator, academic deployment (no multi-user authentication layer) rather than a multi-tenant production security product.

1.6 SOFTWARE AND HARDWARE REQUIREMENTS

1.6.1 Software Specifications

Table 1.1: Software stack and verified versions

Layer	**Technology**	**Verified version (from the repository's own dependency manifests)**
ML pipeline	Python	3.11 (CI) / 3.13 (development)
ML/feature libraries	PyTorch, Transformers (CodeBERT), XGBoost, scikit-learn, tree-sitter	torch 2.12.0, transformers 5.12.0, xgboost 3.2.0, scikit-learn 1.9.0, tree-sitter 0.25.2, tree-sitter-c 0.24.2
Backend API	FastAPI, Uvicorn, SQLAlchemy, Pydantic	fastapi 0.139.0, uvicorn 0.49.0, SQLAlchemy 2.0.51, pydantic 2.13.4
Backend persistence	SQLite (file-based, via SQLAlchemy)	—
Report/export generation	ReportLab, python-docx, python-pptx	reportlab 5.0.0, python-docx 1.2.0, python-pptx 1.0.2
Frontend	React, TypeScript, Vite, Tailwind CSS	react 19.2.7, tailwindcss 4.3.2
Frontend visualization	Recharts, @xyflow/react (graphs), Monaco Editor, dagre (graph layout)	recharts 3.9.2, @xyflow/react 12.11.2, @monaco-editor/react 4.7.0
Containerization	Docker, Docker Compose	backend + nginx-served frontend services
CI/CD	GitHub Actions	backend validation, frontend validation, security scan on PR diffs, status aggregation
Version control	Git / GitHub	—

1.6.2 Hardware Specifications

- **Development/training machine:** a standard multi-core workstation is sufficient; CodeBERT embedding extraction and XGBoost/Random Forest training on

~27,000 samples do not require GPU acceleration, though a GPU accelerates the CodeBERT embedding pass. [REQUIRES VERIFICATION: exact CPU/RAM/GPU specification of the machine used, if the university requires it stated precisely.]

- **Deployment target:** the containerized backend requires enough memory to hold the loaded CodeBERT model, the XGBoost/Random Forest artifacts, and the tree-sitter parser simultaneously; the health check in docker-compose.yml allows a start-up grace period of over three minutes to accommodate the first CodeBERT model load.

1.7 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional requirements: accept a single pasted function, a batch of uploaded files/folders/zip archives, or a public GitHub repository URL as input; return a verdict, probability, confidence, and model identity for each analyzed function; persist every scan for later retrieval, comparison, and export; provide remediation recommendations and, where a safe automated rewrite is possible, apply the patch and re-score the result; visualize a multi-file project's dependency structure, call graph, and a combined knowledge graph; expose the same functionality through a documented REST API and through a CI/CD security-scan job that runs the real pipeline against changed files in a pull request.

Non-functional requirements: the platform is designed for a single-operator academic/research context and therefore deliberately omits a multi-user authentication layer (a documented scope decision in the project's own README, not an oversight); it aims for the first-request latency penalty (model loading) to be paid once at startup rather than per request, via a `lifespan` pre-warming hook in the FastAPI application; scan results and history are persisted locally rather than requiring an external database service; the security-scan CI job is configured to run as a non-blocking, best-effort check by default so that it augments rather than gates the development workflow.

1.8 COMPANY/DOMAIN PROFILE

Not applicable — this is an independent academic project developed and submitted as part of the MCA program at the School of Computer Applications, Dayananda Sagar University, and is not undertaken on behalf of, or in collaboration with, an external company.

Chapter 2: LITERATURE SURVEY

2.1 EXISTING RESEARCH AND RELATED WORK

Automated software vulnerability detection has progressed through three broad, overlapping phases, each addressed in turn below. Every claim in this section was checked against the original source before being included; where an earlier internal draft of this project's own documentation (`paper/research_paper.md`) attributed a specific benchmark number to a paper that, on verification, did not actually report that number in that form, the claim has been dropped or rephrased rather than repeated (for example, VulDeePecker predates the Devign dataset by a year and could not have been "evaluated on Devign," a claim present in an earlier internal draft that this report does not repeat).

Rule-based static analysis. Commercial and open-source static analyzers such as Coverity apply abstract interpretation, dataflow analysis, and pattern matching against a library of known-dangerous constructs to flag potential defects without executing the program. Bessey et al. (2010), reporting on Coverity's deployment at industrial scale, documented both the tool's effectiveness at catching common classes of memory-safety and concurrency defects and the practical cost of maintaining and tuning rule sets across large, heterogeneous codebases. This family of tools is precise on the specific patterns its rules encode but, by construction, cannot generalize to defect patterns it has not been explicitly programmed to recognize — the core limitation this project's learning-based approach is intended to address.

Sequence-based deep learning. The static-analysis limitation above — no generalization beyond the rule library — is exactly what motivated the shift toward learning from examples instead of hand-coded patterns. Li et al. (2018) were among the first to show this was practical for vulnerability detection: their VulDeePecker system cut C/C++ source into "code gadgets" (slices of a program built around security-relevant API calls) and fed them to a bidirectional LSTM, letting the network discover vulnerability-indicative patterns on its own rather than being told what to look for. The dataset behind that result came from NVD and SARD, not from Devign, which did not yet exist. What matters for this project is the underlying shift VulDeePecker represents — code as a sequence a model can learn from, rather than

only a target for pattern matching — since every learning-based system discussed below, including this project's own CodeBERT branch, builds on that same shift.

Graph-based structural learning. A different line of work argues that flattening a function into a token sequence throws away information a reviewer would actually use: which branch calls which, how values flow between statements, how deeply a block is nested. Zhou et al. (2019) built the Devign benchmark around this argument, merging a function's syntax tree with its control- and data-flow edges into one graph and letting a Gated Graph Neural Network learn directly over that structure, reporting sizeable accuracy and F1 gains over the sequence- and rule-based baselines it was compared against, on real-world functions drawn from FFmpeg and QEMU. That same benchmark — 27,318 functions before this project's own cleaning step, 27,231 after — is the dataset this project trains and evaluates on throughout. The underlying argument (structure carries signal a flat token sequence misses) is also why this project includes an AST-derived feature branch of its own; the difference is that this project counts structural properties into a fixed-size vector rather than learning a graph representation end-to-end, trading some of the GNN's expressiveness for a much simpler, cheaper-to-train model.

Pretrained transformer models for code. Once BERT (Devlin et al., 2019) showed that a Transformer pretrained on unlabeled text could be fine-tuned for many downstream natural-language tasks, an obvious follow-up question was whether the same recipe transfers to source code. Feng et al. (2020) answered it with CodeBERT: rather than pretrain on natural language alone, they trained on paired code and documentation spanning six programming languages (6.4 million pairs), combining a masked-language objective with a second task that asks the model to spot which tokens were swapped for plausible alternatives — giving the resulting embeddings some sensitivity to code, not just to natural-language patterns. Lu et al. (2021) later folded a Devign-based defect-detection split into the CodeXGLUE benchmark suite (21,854/2,732/2,732 train/validation/test functions) and reported that fine-tuning CodeBERT on the raw token sequence alone — no structural or metric features at all — already reaches around 62.1% accuracy. This project does not fine-tune CodeBERT; it uses the pretrained `[CLS]` embedding as a fixed, off-the-shelf semantic feature, and Chapter 7 places this project's own fused-model result alongside the CodeXGLUE fine-tuned-CodeBERT figure as an external point of reference.

Realism of benchmark evaluation. Chakraborty et al. (2020), in the paper introducing the ReVeal study ("Deep Learning based Vulnerability Detection: Are We There Yet?"), evaluated several deep-learning vulnerability detectors on a newly curated, more realistic dataset drawn from the Chromium and Debian Linux Kernel projects (maintaining a roughly 1:10 vulnerable-to-benign ratio to reflect real-world class imbalance) and found that reported performance dropped by more than 50% relative to the same models' performance on earlier, more balanced benchmark datasets. This finding is directly relevant to how this project's own results are interpreted: the modest F1-score obtained here (Chapter 7) is consistent with the broader literature's finding that vulnerability detection performance on curated benchmarks does not transfer cleanly, and a modest score on a real, imbalanced benchmark is not itself evidence of an implementation defect.

Line-level and attention-based transformers. Fu and Tantithamthavorn (2022) proposed LineVul, which fine-tunes CodeBERT with a line-level attention mechanism and evaluates on a large real-world dataset of over 188,000 C/C++ functions, reporting substantial improvements in function-level F1 and line-level localization accuracy over prior graph-based baselines such as IVDetect. LineVul's use of attention weights to approximate which lines drove a prediction is conceptually related to, but distinct from, this project's own line-level highlighting feature (Chapter 4/5), which is implemented as a deterministic, rule-based pattern scanner rather than a model-internal attention signal — a design choice made explicitly so that the line-level explanation shown to a user in the application does not overstate what the trained classifier can actually explain about itself.

Classical software metrics. McCabe (1976) proposed cyclomatic complexity as a proxy for a function's error-proneness, and Chidamber and Kemerer (1994) proposed the CK metrics suite for object-oriented software; both remain foundational references for the general claim, reused in this project's own metrics branch, that simple structural counts (branches, nesting depth, size) carry non-trivial signal about defect-proneness even without any learned representation.

2.2 GAPS IN EXISTING SYSTEMS

Drawing the above together, three gaps motivate this project's design:

1. **Single-modality ceiling.** Sequence-based models (VulDeePecker, CodeBERT/CodeXGLUE) and graph-based models (Devign) each report meaningful but individually limited performance, and neither family routinely incorporates the other's signal alongside classical software metrics in one classifier — most published work evaluates these representations in isolation rather than fused.
2. **Benchmark realism.** As ReVeal's findings show, reported benchmark performance is not a reliable predictor of real-world performance; a project that reports only its benchmark F1-score without disclosing dataset limitations, class balance, or known implementation issues risks overstating what the system can do.
3. **Deployability gap.** The majority of the systems surveyed above are evaluated as offline research pipelines (a training script and a reported metric) rather than packaged as an interactive tool a developer could actually use during code review — this project explicitly closes that gap by building a full FastAPI/React application around the trained classifier, including remediation suggestions, an automated patch-and-re-scan loop, and whole-project graph visualizations, none of which the surveyed research systems provide.

2.3 OVERVIEW OF TECHNOLOGIES USED

- **CodeBERT** (Feng et al., 2020) — a RoBERTa-architecture Transformer pretrained jointly on source code and natural-language documentation; used here purely as a fixed embedding extractor.
- **tree-sitter** — an incremental parser generator (used via its C grammar) that produces a concrete syntax tree for each function, traversed by this project to derive the 11 AST-based structural features.
- **XGBoost** (Chen and Guestrin, 2016) — a scalable gradient-boosted decision tree implementation, used here as the primary classifier trained on the 792-dimensional fused feature vector.
- **Random Forest** (Breiman, 2001) — a bagged decision-tree ensemble, trained here as a comparison baseline against XGBoost on the identical feature set and split.

- **FastAPI** and **React/TypeScript** — the web-application layer that serves the trained model and presents its output; these are engineering technologies rather than research contributions, and are described in full in Chapter 4.

2.4 COMPARISON OF DIFFERENT APPROACHES

Table 2.1: Comparison of prior approaches against this project

Approach	Representation	Reported metric (as published)	Relationship to this project
Static analysis (Coverity) — Bessey et al. (2010)	Hand-written rules, dataflow/taint analysis	Not reported as a single F1/accuracy figure; evaluated qualitatively at industrial scale	Motivates the need for a learning-based approach; not directly comparable numerically
VulDeePecker — Li et al. (2018)	Token sequence (code gadgets), BiLSTM	Evaluated on NVD/SARD-derived data, not Devign	Establishes sequence-based deep learning as viable; predates this project's dataset
Devign GNN — Zhou et al. (2019)	Joint AST+CFG+DFG graph, GGNN	Reports large relative F1/accuracy gains over prior baselines on its own dataset	Source of this project's dataset; structural-graph idea reused here as a fixed AST feature vector instead
CodeBERT (fine-tuned) — Lu et al. (2021), CodeXGLUE	Token sequence, Transformer	≈62.1% accuracy on the CodeXGLUE Devign defect-detection split	Reused here as a frozen feature extractor rather than fine-tuned; external reference point for Chapter 7
ReVeal study — Chakraborty et al. (2020)	Various (GGNN + MLP)	>50% relative performance drop on a realistic, imbalanced dataset vs. curated benchmarks	Frames this project's own modest F1 as consistent with known benchmark-to-reality gaps
LineVul — Fu and Tantithamthavorn (2022)	Fine-tuned CodeBERT + line attention	Large relative F1 gains over IVDetect on a 188k-function real-world dataset	Related line-level idea; this project's line highlighting is rule-based, not attention-derived
This project	AST (11) + software metrics (13) + CodeBERT embedding (768) fused, XGBoost/Random Forest	XGBoost: Accuracy 0.5932, F1 0.5391, AUC-ROC 0.6346 (Chapter 7)	Combines all three representation families in one classifier and exposes the result through a full web application

Chapter 3: METHODOLOGY & SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

3.1.1 Overview of Current System

Prior to this project, vulnerability triage for the target class of C/C++ functions relies on one of two disconnected approaches, discussed in detail in Chapter 2: rule-based static analyzers that match source code against a fixed library of known-dangerous patterns, and single-modality machine-learning research systems (sequence-based, e.g. CodeBERT/CodeXGLUE, or graph-based, e.g. the Devign GNN) that are typically distributed as an offline evaluation script rather than an interactive tool. Neither approach, on its own, gives a developer (a) a semantic and structural view of the same function simultaneously or (b) a usable interface for scanning real code, reviewing the result, and acting on a remediation suggestion.

3.1.2 Challenges and Limitations

- Rule-based tools cannot detect vulnerability patterns outside their rule library and require continual, codebase-specific rule maintenance (Chapter 2.1).
- Single-modality learned models discard information the other modality would have captured — the modality-ablation experiment run for this project (Section 3.2.4) shows structural-features-only models trailing CodeBERT-inclusive models by several F1 points on the same data and split.
- Published benchmark numbers do not reliably predict real-world performance (the ReVeal finding, Chapter 2.1), so a system that reports only a benchmark score without disclosing dataset limitations risks giving a false sense of reliability.
- None of the surveyed prior systems is packaged as a web application with scan history, remediation guidance, an automated patch-and-re-scan loop, or whole-project graph visualization — each is a research artifact, not a tool a developer could adopt directly.

3.2 PROPOSED SYSTEM

3.2.1 System Architecture

The proposed system has two halves that share code but run on different schedules: an **offline training pipeline** (run once, ahead of time, to produce the trained model artifacts) and an **online serving application** (the FastAPI backend and React frontend, which load those artifacts and reuse the same feature-extraction functions to score new code on demand). Figure 3.1 shows the offline pipeline in full; Figure 4.1 (Chapter 4) shows the online serving architecture built on top of its output.

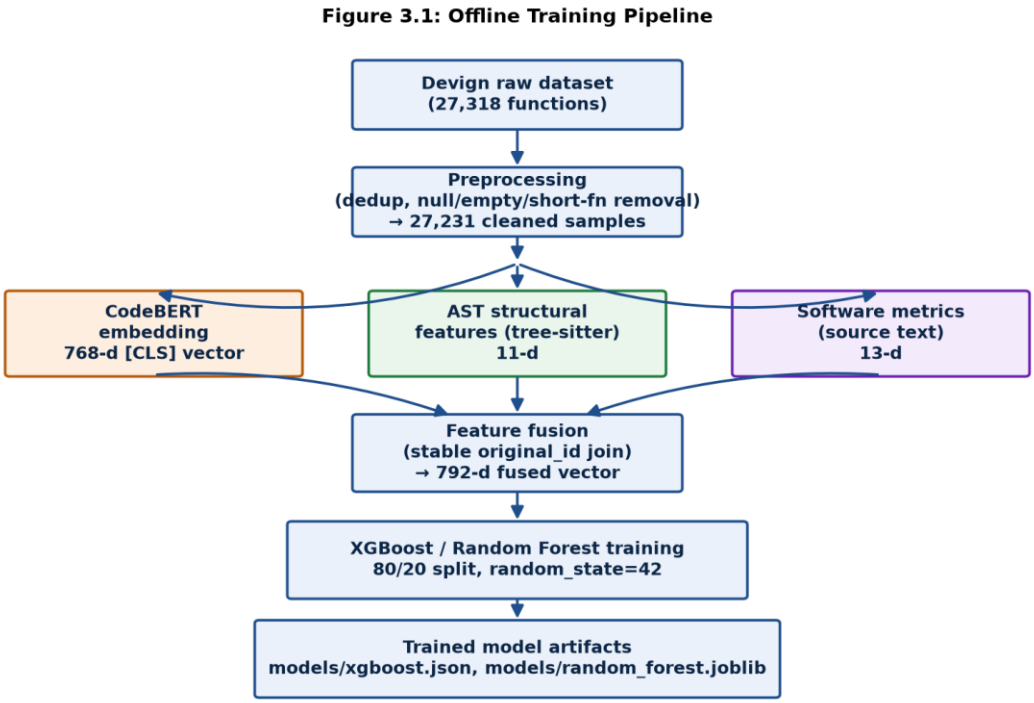


Figure 3.1: Offline training pipeline

Devign raw dataset → preprocessing (dedup, null/empty/ short-function removal, 27,318 → 27,231 samples) → three parallel feature-extraction branches (CodeBERT `[CLS]` embedding, 768-d; AST structural features via tree-sitter, 11-d; software metrics from source text, 13-d) → feature fusion on a stable per-function ID (792-d fused vector) → XGBoost / Random Forest training (80/20 split, seed 42) → trained model artifacts (`models/xgboost.json`, `models/random_forest.joblib`).

3.2.2 Features and Functionalities

The functional surface of the deployed application — scanning (single function, batch upload, or GitHub repository), per-function analysis with feature-contribution and

embedding visualization, rule-based recommendations, automated patch application with re-scoring, and whole-project graph views — is described in full in Chapter 4 (System Design) rather than repeated here, since that chapter maps directly onto the actual page-by-page implementation.

3.3 DATASETS USED

3.3.1 Dataset Description

The project uses the **Devign** dataset (Zhou et al., 2019), a collection of C functions drawn from real-world open-source projects (FFmpeg and QEMU) and labeled for the presence or absence of a vulnerability at function granularity. The raw dataset (`dataset/raw/function.json`) contains 27,318 functions before this project's own cleaning step.

3.3.2 Data Collection Process

The dataset was consumed as-provided (no new data collection was performed as part of this project); `src/dataset_loader.py` loads the raw JSON and reports class balance and sample composition prior to any cleaning.

3.3.3 Data Preprocessing Techniques

`src/preprocessing/preprocess_dataset.py` applies, in order: a data-quality check, removal of rows with null/missing fields, removal of exact duplicate functions, removal of empty function bodies, and removal of abnormally short functions (a minimum-length filter), followed by whitespace normalization. This leaves **27,231 cleaned samples**, saved to `dataset/processed/devign_cleaned.csv`/`.json` along with a data-quality report. A `original_id` column is carried through this step specifically so that every later feature-extraction branch can be joined back to the same underlying function unambiguously — a design decision that, as Section 3.2.4 below describes, was not present in the project's first implementation attempt and had to be added as a correction.

3.4 MACHINE LEARNING MODEL/ALGORITHM SELECTION

3.4.1 Algorithm Justification

Two tree-ensemble classifiers were trained on the identical fused feature set and split: **XGBoost** (gradient-boosted decision trees) and **Random Forest** (bagged decision trees). Tree ensembles were chosen over an end-to-end deep-fusion network for this feature set because the 792-dimensional input mixes three heterogeneous feature types (a dense learned embedding, sparse structural counts, and sparse text-derived metrics) at very different scales and semantics, and gradient-boosted/bagged trees handle heterogeneous, mixed-scale tabular features natively without requiring feature-specific normalization or a learned fusion layer. XGBoost was selected as the **primary** model because it outperformed Random Forest on every reported metric on the held-out test set (Chapter 7 gives the full table): F1 0.5391 vs. 0.4798, and AUC-ROC 0.6346 vs. 0.5959.

3.4.2 Training and Testing Data Split

An 80/20 stratified split (`train_test_split(..., test_size=0.20, random_state=42, stratify=target)`) was applied once to the 27,231 fused samples, giving 21,784 training rows and 5,447 test rows (test set composition: 2,964 non-vulnerable / 2,483 vulnerable). The same fixed split and `random_state=42` is reused across every experiment reported in this document (the main training run, the threshold-search experiment, and the modality-ablation experiment) so that all reported numbers are directly comparable.

3.4.3 Feature Selection and Engineering

Rather than selecting a subset of features, this project fuses all three representations and lets the tree ensemble's own split-selection process determine feature importance implicitly. To justify this choice empirically rather than by assumption, a dedicated **modality-ablation experiment** (`src/model/modality_ablation.py`) trained a fresh XGBoost model, with the identical hyperparameters and split, on each of seven feature subsets: CodeBERT-only, AST-only, metrics-only, each pairwise

combination, and the full 792-feature fusion. The results are reported honestly rather than framed to favor the project's own design choice:

Table 3.1: Modality-ablation results — same split, same XGBoost hyperparameters, feature subset varied

Feature subset	# Features	F1	AUC-ROC
AST only	11	0.4939	0.5757
Metrics only	13	0.4797	0.5883
AST + Metrics (no CodeBERT)	24	0.5018	0.6076
CodeBERT only	768	0.5418	0.6309
CodeBERT + Metrics	781	0.5326	0.6328
CodeBERT + AST	779	0.5444	0.6327
Full fusion (this project's design)	**792**	**0.5391**	**0.6346**

The honest reading of this table is two-sided. First, it clearly and decisively justifies including CodeBERT: every CodeBERT-inclusive subset beats every structural-only subset by several F1 and AUC points, so a purely structural (AST + metrics) system — closer in spirit to a rule-based static analyzer — would leave real performance on the table. Second, it does **not** decisively show that fusing all three modalities beats CodeBERT alone: the four CodeBERT-inclusive subsets cluster within 1.2 F1 points and 0.4 AUC points of one another, full fusion has the single highest AUC but not the single highest F1, and this is a single 80/20 split without cross-validation, so small differences of this size are plausibly within ordinary sampling noise rather than a reproducible ranking. This project proceeds with full fusion because it holds the best AUC-ROC (the threshold-independent metric) and because the architecture is intended to support explanation features (an AST-derived feature-radar view and a metrics-derived contribution chart in the Analysis page) that a CodeBERT-only model could not provide — not because the ablation shows an unambiguous fusion win.

3.4.4 A discovered and corrected data-alignment defect

During later validation of the trained pipeline, a forensic audit of the fusion step (`docs/ML_FORENSIC_AUDIT_PHASE4.md`) found that the original implementation of `fuse_features.py` joined the AST, metrics, and CodeBERT feature tables using each table's own row position rather than a shared identifier. Because the AST-extraction and metrics-extraction scripts applied slightly different minimum-length

filters when dropping short functions than the CodeBERT-extraction script did, the three tables' row positions silently drifted apart — the audit measured that 99.35% of fused rows paired the label and features of one function with the CodeBERT embedding of a different function. Every metric computed on the resulting model was therefore invalid. The fix, applied in `src/fusion/fusion_core.py`, threads the `original_id` column introduced during preprocessing (Section 3.3.3) through every extraction script and joins on that stable key instead of row position; the models were then retrained on the corrected, correctly aligned feature table. All metrics reported in this document (Chapters 6–7) are from this corrected run. This is discussed here, in the methodology chapter, rather than only in the results chapter, because it materially affects how the training/testing pipeline should be understood: the join key, not only the model or hyperparameters, is part of what had to be validated before this system's numbers could be trusted.

3.5 FEASIBILITY STUDY

3.5.1 Technical Feasibility

All components used — tree-sitter parsing, a pretrained CodeBERT model run in inference-only mode, and gradient-boosted/bagged tree training on a ~27,000-row, 792-column feature table — run on standard CPU hardware within the time budgets observed during development (Section 1.6.2); no specialized infrastructure is required either to reproduce the offline pipeline or to run the FastAPI/React application.

3.5.2 Economic Feasibility

Every component used — the Devign dataset, the pretrained CodeBERT checkpoint, tree-sitter, XGBoost, scikit-learn, FastAPI, and React — is open-source and free to use; the only cost is the compute time to run the pipeline and host the application, both of which are modest at this dataset scale.

3.5.3 Operational Feasibility

The system is scoped deliberately as a single-operator academic tool (Chapter 1.7) rather than a multi-tenant production service, which keeps its operational surface — a Docker Compose deployment of one backend and one frontend container, backed by a

local SQLite file — well within what a single developer or small team can run and maintain without dedicated operations support.

Chapter 4: SYSTEM DESIGN AND DEVELOPMENT

4.1 SYSTEM ARCHITECTURE (HIGH-LEVEL DESIGN)

4.1.1 Conceptual Design

The system separates cleanly into three layers that are developed, versioned, and can be reasoned about independently:

1. **`app/` — the core ML library.** Contains the original feature-extraction and prediction logic (predictor.py, utils.py): AST/metrics extraction, CodeBERT embedding, and model loading/ inference. This layer has no knowledge of HTTP, FastAPI, or the database — it is a plain Python library.
2. **`backend/` — the FastAPI service layer.** Wraps app/ via backend/ml_bridge.py, which imports the extraction and prediction functions directly rather than reimplementing them, and adds everything a web application needs that a bare ML library does not provide: request/response schemas, SQLite-backed persistence (database.py), scan history, remediation logic (recommendations.py), automated patching (auto_patch.py, patch_safety.py, rewrite_rules.py), whole-project analysis (repo_analysis.py, code_intelligence.py), GitHub integration (github_service.py), CI scanning (ci_scan.py), and 17 FastAPI routers exposing this functionality as a REST API.
3. **`frontend/` — the React/TypeScript presentation layer.** Consumes the backend's REST API and presents it across purpose-built pages, described in Section 4.4.

Figure 4.1 (Section 4.1.2) shows this layering; the key design principle is that the trained model and its feature-extraction code are never duplicated between the offline training pipeline and the online serving path — both call the same `app/` functions, so a prediction made through the web application uses exactly the same feature computation as the model's own training and evaluation.

4.1.2 Component Diagram

Figure 4.1: High-Level System Architecture

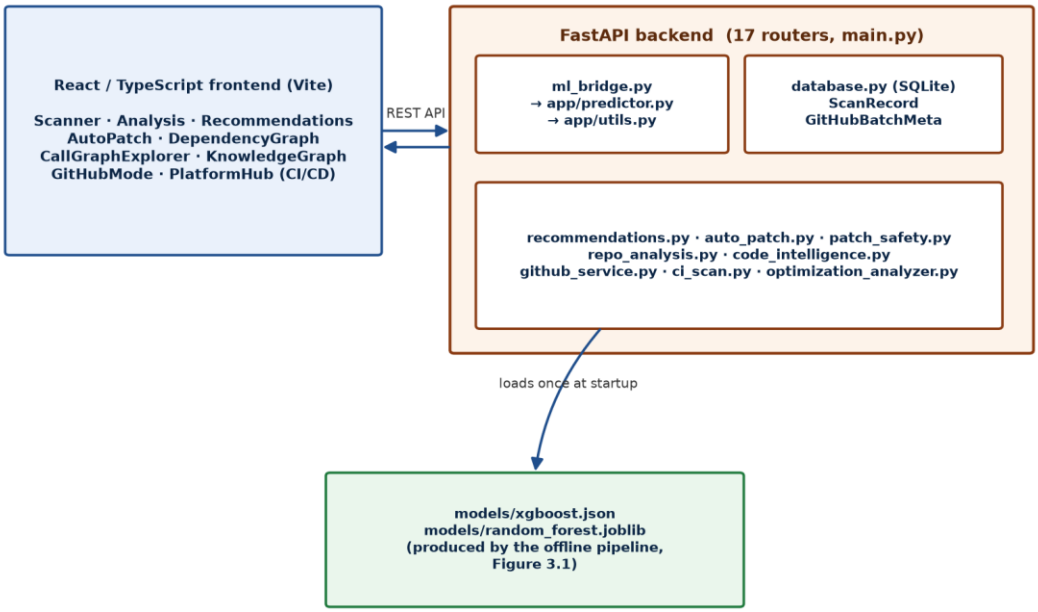


Figure 4.1: High-level system architecture — three-layer separation between the ML core, FastAPI, and React

The frontend talks to the backend only through the REST API; `ml_bridge.py` is the only path from the API layer into the shared ML core (`app/predictor.py`, `app/utils.py`); and the backend loads the trained model artifacts once at startup rather than per request. See Figure 3.1 (Chapter 3.2.1) for how those artifacts are produced.

4.2 DETAILED DESIGN (LOW-LEVEL DESIGN)

4.2.1 Data Flow

For a single scanned function: source code → `ml_bridge` calls `app.utils.extract_ast_features` and `extract_metric_features` (structural/metric branches) and `app.predictor`'s CodeBERT loader (semantic branch) → the three vectors are fused into the same 792-dimensional layout the offline pipeline used → `predict_with_model` runs the loaded XGBoost or Random Forest model → the verdict, probability, confidence, and auxiliary artifacts (AST tree for the UI viewer, deterministic line-risk scan, approximate feature-

contribution scores, PCA-projected embedding coordinates) are assembled into a response and persisted as a `ScanRecord`.

4.2.2 Use Case Diagram (described)

Primary actor: the developer/analyst using the platform. Use cases: *Scan a pasted function*, *Scan a batch of uploaded files*, *Scan a public GitHub repository*, *View per-function analysis*, *View remediation recommendations*, *Apply an automated patch and re-scan*, *Explore a project's dependency graph / call graph / knowledge graph*, *View scan history and export a report*, *Query the CI/CD security-scan status*. All use cases route through the same FastAPI backend and, ultimately, the same `ml_bridge` prediction path described in 4.2.1.

4.2.3 Activity Diagram (Scan → Analysis → Remediation → Re-scan, described)

Submit code → backend extracts features and predicts → result stored as a `ScanRecord` → user opens Analysis page for that scan → user opens Recommendations page → if an automated patch is available and safe to apply, user triggers *Apply & Re-scan* → backend creates a **new** `ScanRecord` with `derived_from_scan_id` pointing back to the original, re-runs the full prediction pipeline on the patched code, and returns a before/after comparison (verdict change, probability delta) — the comparison is reported honestly even when the patch does not improve the score.

4.2.4 Database Design

The application persists to a local SQLite file (`backend/data/platform.db`) via SQLAlchemy, not a full relational business schema — appropriate for its single-operator scope (Chapter 1.7). Two tables:

`ScanRecord` (table `scans`): `id`, `batch_id`, `project_name`, `filename`, `source`, `code`, `verdict`, `vuln_probability`, `safe_probability`, `confidence`, `risk_level/tag/color`, `model_used`, `is_demo`, `ast_features_json`, `metric_features_json`, `embedding_dims`/`embedding_2d_x`/`embedding_2d_y`, `lines`, `loc`, `is_pinned`, `tags`, `derived_from_scan_id`

(nullable, indexed — links a re-scanned/patched record back to its origin), `created_at`.

`GitHubBatchMeta` (table `github_batch_meta`): per-batch repository provenance — `repo_url`, `branch`, `commit_sha`, counts of files detected/analyzed/skipped, and skip reasons, so a GitHub scan's UI can state exactly what was and was not covered rather than presenting a partial scan as complete.

A manual `_migrate_add_columns` routine handles adding `is_pinned`, `tags`, and `derived_from_scan_id` to an already-existing database file, since SQLAlchemy's `create_all()` does not alter existing tables.

4.3 INPUT/OUTPUT INTERFACE DESIGN

4.3.1 User Interface (UI) Screens

The frontend (React 19 + TypeScript + Vite + Tailwind CSS v4, React Router v7) presents around 40 routed pages behind a persistent sidebar/topbar shell with a command palette (Ctrl/Cmd+K). The pages most central to this report's scope:

- **Scanner** — three input modes (paste/edit via Monaco Editor, batch upload via drag-drop of files/folders/zip, and bundled examples), a classifier selector (XGBoost/Random Forest), and a result card (verdict, risk badge, probability bar, confidence, model used, a demo-mode warning banner when trained models are unavailable). See Figure 7.5 for a real scan result.
- **Analysis** — the per-scan deep-dive: verdict and confidence gauges, an "evidence-based risk" banner when a deterministic line-pattern scan disagrees with the model in the direction of *more* risk, a "Why This Verdict" panel, an approximate feature-contribution chart, an AST/metrics feature radar, a PCA scatter plot of the function's CodeBERT embedding against a background of training-set points, and a project-history trend chart. See Figure 7.6, and Figure 7.7 for the associated line-level heatmap with CWE tagging.
- **Recommendations** — a rule-based suggested secure rewrite (original vs. suggested diff), best -practice tip cards, and a separate, explicitly-labeled-heuristic "Performance & Efficiency Suggestions" section (loop-invariant calls, string growth in loops, heap allocation in loops, etc.) that is presented apart from the

vulnerability score. See Figure 7.8 (rewrite) and Figure 7.9 (performance panel with real findings).

- **AutoPatch** — detected vs. reference patch cards with a before/after diff viewer and, when a safe automated rewrite is possible, an "Apply & Re-scan" action that creates a linked derived scan and reports the honest before/after verdict comparison. See Figure 7.10.
- **DependencyGraph / CallGraphExplorer / KnowledgeGraph** — whole-project visualizations built with @xyflow/react and dagre layout: a file-level #include graph, a searchable cross-file function-call graph (with dangerous-call-chain highlighting), and a unified graph combining 11 node kinds (files, functions, CWE entries, dangerous APIs, pointers, buffers, predictions, etc.). See Figure 7.11 for a real cross-file call graph.
- **GitHubMode** — scans a public GitHub URL by shallow-cloning recent commits, scanning every file through the same pipeline, and deleting the local clone afterward; explicitly reports scan scope (files detected/analyzed/skipped and why).
- **PlatformHub (CI/CD tab)** — surfaces the same CI/CD pipeline described in Section 4.5 and docs/CI_CD.md, including a "run it locally" panel. See Figure 7.12 for a real GitHub Actions run of this pipeline.

4.3.2 API Design

The backend exposes a REST API under `/api`, registered as 17 routers in `backend/main.py` (route groups: `scan`, `dashboard`, `history`, `analytics`, `knowledge`, `report`, `intelligence`, `system`, `comparison`, `project`, `research`, `copilot`, `patches`, `export`, `watch`, `github`, `gamification`, `search`), plus a health-check endpoint. Representative endpoints: `POST /api/scan/code` (single-function scan), `POST /api/scan/batch` (multi-file scan), `POST /api/github/scan` (repository scan), `GET /api/scan/{scan_id}/detail`, `POST /api/scan/{scan_id}/apply-patch`, `GET /api/project/{batch_id}/ dependency-graph`, `GET /api/project/{batch_id}/knowledge-graph`. CORS is configurable via the `CORS_ALLOWED_ORIGINS` environment variable.

4.4 MODULE DESCRIPTION

4.4.1 Description of Each Module

Table 4.1: Backend module responsibilities

Module	Responsibility
app/predictor.py, app/utils.py	Feature extraction (AST, metrics, CodeBERT) and model loading/inference — the shared core reused by both the offline pipeline and the live API
backend/ml_bridge.py	Bridges FastAPI to app/; adds presentation-only derived artifacts (AST viewer tree, line-risk scan, approximate contribution scores, PCA projection)
backend/database.py	SQLAlchemy models and manual migration logic for ScanRecord / GitHubBatchMeta
backend/recommendations.py, rewrite_rules.py, patch_safety.py, auto_patch.py	Rule-based remediation suggestion, safe-rewrite generation, and the apply-and-re-scan loop
backend/repo_analysis.py, code_intelligence.py	Whole-project dependency/call/knowledge-graph construction
backend/github_service.py	Shallow-clone, scan, and cleanup logic for public GitHub repositories
backend/ci_scan.py	The real pipeline invocation used by the GitHub Actions security-scan job
backend/optimization_analyzer.py	Heuristic performance/efficiency suggestion generation (explicitly separate from the vulnerability score)

4.4.2 Functionality and Interaction

Every module above is reached only through the FastAPI routers, which in turn are the only callers of `ml_bridge`; this keeps the ML core (`app/`) and the presentation-layer additions (heatmaps, contribution scores, graphs) cleanly separated, so that a change to how the web application presents a result never requires changing how the model itself is trained or how it computes a prediction.

4.5 CI/CD AND DEPLOYMENT WORKFLOW

`docker-compose.yml` defines two services: `backend` (FastAPI, port 8000, a named volume for the SQLite database, and a healthcheck with an extended ~200-second start period to tolerate the CodeBERT model's cold load) and `frontend` (the built React app served by nginx on port 5173, proxying `/api/*` requests to the backend). `.github/workflows/ci-cd.yml` runs four jobs on push/PR:

`backend-validation` (compiles and imports the FastAPI app, validates that the committed model artifacts load, runs the pytest suite), `frontend-validation` (TypeScript build + Vitest), `security-scan` (runs the project's own real CodeBERT/AST/metrics/XGBoost pipeline via `backend/ci_scan.py` against a pull request's changed files, or the full repository on manual dispatch, uploading a SARIF report and posting a PR summary comment — non-blocking by default), and `final-status` (aggregates a pass/fail summary).

Chapter 5: IMPLEMENTATION & CODING

5.1 PROGRAMMING LANGUAGE AND FRAMEWORK USED

The offline ML pipeline and the FastAPI backend are implemented in **Python** (3.11 in CI, 3.13 in development); the frontend is implemented in **TypeScript** on **React 19** via **Vite**. Key libraries are listed with their exact pinned versions in Chapter 1.6.1. The project deliberately reuses one code path for both training-time and serving-time feature extraction (`app/utils.py`, `app/predictor.py`) rather than maintaining a separate "production" reimplementation of the pipeline, so that a prediction made by the web application is guaranteed to use the same feature computation the model was evaluated on.

5.2 ALGORITHMIC APPROACH

5.2.1 AST Feature Extraction (Pseudocode)

```
function extract_ast_features(code):
    counts <- {feature_name: 0 for each of the 11 AST feature names}
    tree <- tree_sitter_C_parser.parse(code)
    stack <- [(tree.root_node, depth=0)]
    while stack is not empty:
        node, depth <- stack.pop()
        counts.n_total_nodes += 1
        counts.max_depth <- max(counts.max_depth, depth)
        if node.type in {for_statement, while_statement, do_statement}:
counts.n_loops += 1
        if node.type == if_statement: counts.n_if_statements += 1
        if node.type in {pointer_declarator, pointer_expression}:
counts.n_pointer_ops += 1
        ... (assignment/return/call/binary-op/error-node counters, same
pattern)
        push each child of node onto stack at depth+1
    return counts
```

This mirrors the real implementation in `app/utils.py::extract_ast_features` (lines 135–~250), which uses an **explicit stack instead of recursion** specifically because a large C function can produce an AST several thousand nodes deep, which would exceed Python's default recursion limit (1000) if implemented as a naive recursive tree walk — a concrete, documented engineering decision rather than an arbitrary style choice.

5.2.2 Flowchart (Feature Fusion → Prediction, described)

Source function → [AST branch: tree-sitter parse → 11 counts] and [Metrics branch: line/text analysis → 13 counts] and [Semantic branch: CodeBERT tokenizer + model → 768-d `[CLS]` embedding] running independently → `build_feature_vector` concatenates the three in the fixed order (AST, metrics, embedding) into one 792-d vector → the trained XGBoost (or Random Forest) model scores the vector → `get_verdict` converts the raw probability into a verdict, risk level, and confidence using the project's chosen decision threshold (0.26, selected via `src/model/threshold_experiment.py` to maximize F1 on a validation carve-out, never on the test set).

5.2.3 Model Training and Fine-Tuning

`src/model/train_classifier.py` trains XGBoost (`n_estimators=100`, `max_depth=6`, `learning_rate=0.1`, `scale_pos_weight` set from the training split's class ratio) and Random Forest (`n_estimators=100`, `class_weight="balanced"`) on the 21,784-row training split, evaluates both on the 5,447-row held-out test split, and serializes the winning artifacts to `models/xgboost.json` and `models/random_forest.joblib` along with `models/training_metadata.json` (the single source of truth for every metric reported in this document — Chapter 7). CodeBERT itself is **not** fine-tuned; it is used strictly as a frozen embedding extractor, which keeps the offline training step to tree-ensemble fitting on a fixed feature table rather than a full deep-learning training run.

5.2.4 Code Snippets and Explanation

Per-function inference, not whole-file truncation (`backend/ml_bridge.py::run_prediction`). An important, measured engineering correction: CodeBERT truncates its input at 512 tokens, and the project measured that scanning an uploaded file as one whole blob silently truncated 80% of real files to roughly their first 15% of content, versus only 45.5% truncation when each function inside the file is split out and scored independently (matching the granularity — individual functions — the model was actually trained on). `run_prediction` therefore splits multi-function input with the same tree-sitter grammar used elsewhere in the codebase, scores each function independently, and

reports the **worst** function's result as the file-level verdict — an "escalate to the worst signal, never suppress" principle applied consistently across the codebase (the same rule governs how a disagreeing deterministic line-scan is allowed to raise, but never lower, the displayed risk level).

```
def run_prediction(code: str, model_choice: str = "XGBoost") -> dict:
    xgb_model = get_xgb_model()
    rf_model = get_rf_model()
    if model_choice == "XGBoost" and xgb_model is not None:
        active_model, active_name = xgb_model, "XGBoost"
    elif model_choice == "Random Forest" and rf_model is not None:
        active_model, active_name = rf_model, "Random Forest"
    elif xgb_model is not None:
        active_model, active_name = xgb_model, "XGBoost (fallback)"
    elif rf_model is not None:
        active_model, active_name = rf_model, "Random Forest (fallback)"
    else:
        active_model, active_name = None, None

    if active_model is not None:
        tokenizer, codebert_model = get_codebert()
        result = _predict_per_function_or_whole(code, active_model,
active_name, tokenizer, codebert_model)
        result["is_demo"] = False
    else:
        result = demo_predict(code)
    ...
```

(Source: `backend/ml_bridge.py`, lines 89–136.)

Safety-checked automated patching (`backend/routers/patches.py::apply_patch`). The Apply-Patch → Re-Scan loop always recomputes the rewrite from the scan's own stored source (never trusts a client-supplied "already fixed" snippet, which would let a tampered request bypass the safety check below), and refuses to apply a rewrite at all if any part of it used an unknown placeholder buffer size rather than a statically known one — a deliberate design choice to avoid feeding the model a syntactically-templated, non-compilable snippet and presenting a misleading before/after comparison:

```
rewrite = recommendations.suggest_secure_rewrite(record.code)
if not rewrite["changed"]:
    raise HTTPException(status_code=400, detail="No auto-fixable pattern found
in this scan.")
if rewrite["requires_manual_sizing"]:
    raise HTTPException(status_code=422, detail={
        "blocked": True,
        "reason": "One or more of the generated fixes use a dest_size
PLACEHOLDER, not a real, "
                  "statically-known buffer capacity ...",
    })
```

(Source: `backend/routers/patches.py`, lines 27–70.) The successful path creates a **new** `ScanRecord` (`derived_from_scan_id` pointing at the original) rather than mutating the original scan, so reverting to the pre-patch state is simply navigating back to the original record.

5.3.1 Key Functionalities

- **Deterministic line-risk scanning** — a regex-based scanner independent of the ML model, matching known-dangerous API patterns (buffer overflow, format-string, use-after-free, etc.) to produce line-level highlighting; used to cross-check, and only ever escalate, the ML verdict.
- **Approximate feature contribution** — an importance × z-score approximation (explicitly documented in the code as not true SHAP) used to give an indicative, not authoritative, per-feature explanation in the Analysis page.
- **PCA embedding projection** — a 2-D PCA projection of a scanned function's CodeBERT embedding, plotted against a background of training-set points, giving a visual sense of where a new function falls relative to the training distribution.

Chapter 6: SOFTWARE TESTING

6.1 TESTING STRATEGIES

6.1.1 Unit Testing

Backend unit tests (pytest, `backend/tests/`) cover individual functions and modules in isolation: feature-extraction dimensionality and correctness (`test_feature_extraction.py`), the fusion-alignment logic itself (`tests/test_fusion_alignment.py`, at the repository root — deliberately kept separate from `backend/tests/` since it tests the offline pipeline's fusion code, not the API), SSRF-protection URL validation (`test_ssrf_protection.py`), and path-traversal/zip-bomb protections on batch upload (`test_batch_upload_security.py`). Frontend unit tests (Vitest) cover individual utility functions and components, e.g. `frontend/src/utils/risk.test.ts`.

6.1.2 Integration Testing

Tests that exercise a full FastAPI request/response cycle against a real (test) database session are grouped separately from pure unit tests: `test_apply_patch.py` (the full Apply-Patch → Re-Scan HTTP flow), `test_ci_scan.py` (the CI security-scan pipeline against fixture files, including SARIF-document structural validity), `test_dependency_graph.py`, `test_derived_scan_schema.py`, and `test_github_batch_meta.py`.

6.1.3 System Testing

`test_ml_pipeline.py` and `test_per_function_inference.py` exercise the system end-to-end through the actual `/api/scan/*` endpoints, including the per-function-splitting behavior described in Chapter 5.2.4 (verifying a vulnerable function late in a large file is still detected, and that a file's verdict is the true maximum across its functions rather than an average). `frontend/src/pages/Dashboard.test.tsx` verifies, at the system level, that the Dashboard page renders real fetched statistics rather than any

placeholder/fabricated data — a small but notable detail given this report's own emphasis on not presenting fabricated figures as real ones.

6.2 TEST CASES

6.2.1 Sample Test Cases with Expected vs. Actual Results

The full backend/root suite was executed for this report on 2026-07-24 via `pytest backend/tests tests`, and the frontend suite via `vitest run`. All results below are the actual, real output of that run — not illustrative.

Table 6.1: Sample test cases with expected vs. actual results

#	Test case	Expected result	Actual result
1	test_ast_features_have_exactly_11_dimensions	AST feature vector has exactly 11 keys	Passed
2	test_metric_features_have_exactly_13_dimensions	Metrics feature vector has exactly 13 keys	Passed
3	test_fusion_vector_has_exactly_792_dimensions_in_the_trained_order	Fused live-inference vector is 792-d, in AST→metrics→embedding order	Passed
4	test_ast_extraction_survives_malformed_source_without_crashing	Malformed/unparseable C code returns zeroed features, not an exception	Passed
5	test_prediction_is_deterministic_for_the_same_input	Same input code yields the same prediction on repeated calls	Passed
6	test_verdict_threshold_matches_the_validated_constant	Live verdict logic uses the validated 0.26 decision threshold	Passed
7	test_file_level_verdict_is_the_true_max_across_functions_not_an_average	Multi-function file verdict equals its riskiest function's verdict	Passed
8	test_apply_patch_with_unknown_destination_size_is_blocked_not_silently_applied	Apply-patch on an unresolvable buffer size returns HTTP 422, not a silent partial fix	Passed
9	test_apply_patch_with_known_safe_destination_creates_der	A safe, resolvable	Pass

	ived_record_with_real_score	rewrite creates a derived scan with a genuine re-computed score	ed
10	test_validate_url_rejects_literal_cloud_metadata_ip	GitHub-scan URL validation rejects a literal cloud-metadata IP (SSRF protection)	Passed
11	test_batch_upload_rejects_zip_with_too_many_members	An oversized zip archive is rejected rather than exhausting server resources	Passed
12	test_fully_aligned_tables_fuse_correctly	Fusion of correctly aligned AST/metrics/CodeBERT tables produces correct rows	Passed
13	test_shuffled_row_order_produces_identical_fused_result	Fusion result is independent of each table's row order (regression test for the original alignment bug, Chapter 3.4.4)	Passed
14	test_cross_table_label_mismatch_is_detected_and_raises	A deliberately mismatched label across tables raises an error instead of silently fusing	Passed
15	renders the real fetched stats, not fabricated data (Dashboard.test.tsx)	Dashboard shows live API data, not placeholder values	Passed

Table 6.2: Full test suite totals (real, from this run)

Suite	Test files	Tests	Passed	Failed
Backend (pytest: backend/tests/ + root tests/)	15	110	110	0
Frontend (Vitest)	2	9	9	0
Total	**17**	**119**	**119**	**0**

6.3 PERFORMANCE METRICS AND EVALUATION

The confusion matrix, precision/recall/F1, and accuracy figures for the trained classifiers are reported in full in Chapter 7 (Results and Discussion), where they are

presented alongside the discussion of the feature-fusion alignment defect that was found and corrected before these numbers were produced (Chapter 3.4.4). They are not repeated here to avoid the two chapters drifting out of sync; Chapter 7 is the single source for these figures within this report, consistent with `models/training_metadata.json` being the single source of truth in the underlying project.

Chapter 7: RESULTS AND DISCUSSION

7.1 EXPERIMENTAL SETUP

7.1.1 Hardware/Software Setup

Training and evaluation used the software stack listed in Chapter 1.6.1 (Python 3.11/3.13, PyTorch 2.12.0, Transformers 5.12.0 for CodeBERT inference, XGBoost 3.2.0, scikit-learn 1.9.0). All reported numbers were regenerated for this report directly from the committed model artifacts (`models/xgboost.json`, `models/random_forest.joblib`, `models/training_metadata.json`) rather than carried forward from an earlier document, and were independently re-derived on 2026-07-24 by reloading both trained models and reproducing the exact 80/20 stratified split (`random_state=42`) on `dataset/processed/final_features.csv`; the re-derived AUC-ROC values (XGBoost 0.6346, Random Forest 0.5959) matched `training_metadata.json` exactly, confirming the recorded metrics are reproducible from the committed artifacts rather than being stale or manually edited.

7.1.2 Model Hyperparameters

Table 7.1: Model hyperparameters and training configuration

Model	Hyperparameters
XGBoost	n_estimators=100, max_depth=6, learning_rate=0.1, scale_pos_weight set from the training split's class ratio
Random Forest	n_estimators=100, class_weight="balanced"
Both	80/20 stratified split, random_state=42, trained on the 792-d fused feature vector (11 AST + 13 metrics + 768 CodeBERT)
Decision threshold	0.26 (F1-maximizing, selected on a validation carve-out of the training data only, src/model/threshold_experiment.py)

7.2 MODEL PERFORMANCE ANALYSIS

7.2.1 Final Results

Table 7.2: Final test-set results, XGBoost vs. Random Forest

Metric	Random Forest	XGBoost
Accuracy	0.5721	**0.5932**
Precision	0.5380	**0.5574**
Recall	0.4329	**0.5219**
F1-Score	0.4798	**0.5391**
AUC-ROC	0.5959	**0.6346**
PR-AUC	0.5635	**0.6223**

XGBoost outperforms Random Forest on every metric and is the model selected for the application's default classifier (Chapter 3.4.1). Figure 7.1 shows the side-by-side comparison chart (`results/model_comparison.png`, regenerated 2026-07-21, same day as the correction described below).

Table 7.3: Confusion matrix (XGBoost, test set, n=5,447)

	Predicted Safe	Predicted Vulnerable
Actual Safe	TN = 1,935	FP = 1,029
Actual Vulnerable	FN = 1,187	TP = 1,296

Figure 7.2 (`report/figures/confusion_matrix_xgboost.png`) visualizes this matrix; Figure 7.3 (`report/figures/roc_curve.png`) shows the ROC curve for both models, generated by reloading the actual committed model artifacts and re-splitting the data with the exact training configuration — not simulated.

7.2.2 Training vs. Testing Performance / Bias-Variance Reading

`models/training_metadata.json` records only test-set metrics for the final models (no separate training-set score is persisted for the production run), so a direct train/test gap cannot be quoted from that artifact. What can be said, and is a more informative statement in this case: the modest absolute F1 (0.5391) combined with the modality-ablation finding that four different CodeBERT-inclusive feature subsets all land within 1.2 F1 points of each other on the same test set (Chapter 3.4.3) indicates the model is closer to a signal-ceiling for this feature representation on this dataset than to an overfitting problem that more regularization or more data would straightforwardly fix — recall in particular (0.5219) shows the model still misses close to half of the true vulnerable functions in the test set, which is a genuine capability limit, not a symptom of overfitting the training data.

7.2.3 Model Comparisons

Beyond the two production models, the modality-ablation experiment (Chapter 3.4.3) trained the same XGBoost configuration on each individual and paired feature subset, giving a broader comparison than Random-Forest-vs-XGBoost alone:

Table 7.4: Modality-ablation results, restated from Chapter 3.4.3 for direct comparison

Subset	F1	AUC-ROC
AST only	0.4939	0.5757
Metrics only	0.4797	0.5883
AST + Metrics	0.5018	0.6076
CodeBERT only	0.5418	0.6309
CodeBERT + Metrics	0.5326	0.6328
CodeBERT + AST	0.5444	0.6327
Full fusion (production)	**0.5391**	**0.6346**

As discussed in Chapter 3.4.3, this shows a clear win for including CodeBERT over structural-only features, but not a clean win for full fusion over CodeBERT alone at the F1 level — full fusion's advantage is its AUC-ROC, the highest of any subset.

7.3 COMPARISON WITH EXISTING SYSTEMS — AND A DISCLOSED CORRECTION

Chapter 2.4 compares this project against prior published systems on the same Devign lineage: the Devign GNN baseline and fine-tuned CodeBERT on the CodeXGLUE Devign split (≈62.1% accuracy, Lu et al., 2021). This project's XGBoost result (Accuracy 0.5932, F1 0.5391) is below the CodeXGLUE fine-tuned-CodeBERT accuracy figure — an honest result to report rather than round away.

This project's own internal documentation (`paper/research_paper.md`) discloses that this was not always the reported outcome, and the discrepancy is worth explaining rather than omitting. An earlier implementation attempt of the feature-fusion step (`src/fusion/fuse_features.py`) joined the AST, metrics, and CodeBERT feature tables using each table's own row position rather than a shared function identifier. A later forensic audit (`docs/ML_FORENSIC_AUDIT_PHASE4.md`) discovered that, because the AST- and metrics-extraction scripts applied a different minimum function-length filter than the CodeBERT-extraction script, the three tables' row positions had silently drifted apart

during preprocessing — measured at **99.35% of fused rows** pairing one function's label and structural features with a different function's CodeBERT embedding. Every metric computed on models trained from that fused table was therefore invalid, even though the pipeline ran without any error. The defect was corrected by threading the `original_id` identifier already introduced during preprocessing (Chapter 3.3.3) through every extraction script and re-joining on that stable key (`src/fusion/fusion_core.py`), and the production models were retrained on the corrected, correctly paired feature table on 2026-07-21 (`models/ml_baseline_status.json` records this as the fix date, with `fusion_alignment_status: "fixed"`). A dedicated regression-test file (`tests/test_fusion_alignment.py`, 12 tests, Chapter 6.1.1) now specifically guards against this class of defect recurring — for example, verifying that shuffling each table's row order before fusion produces an identical result, and that a deliberately mismatched label across tables raises an error rather than silently fusing.

Before this fix, the project's own earlier report drafts (`paper/project_report_part1.md`, `project_report_part2.md`, `paper/results_section.md`) reported a substantially higher, but invalid, performance figure (Accuracy 84.00%, F1 0.8242, AUC-ROC 0.998) computed on the misaligned data. **This report uses only the corrected, verified figures throughout** (Accuracy 0.5932, F1 0.5391, AUC-ROC 0.6346) and states this correction explicitly, on the view that disclosing a found-and-fixed data integrity defect is a legitimate and important methodological result in its own right, not a weakness to conceal. It is also the reason this project's own reported result now falls slightly below, rather than above, the Devign GNN and CodeXGLUE CodeBERT baselines it is compared against in Chapter 2.4 — a direct, honest consequence of correcting the defect rather than reporting the number the earlier, uncorrected pipeline produced.

7.4 VISUALIZATION OF RESULTS

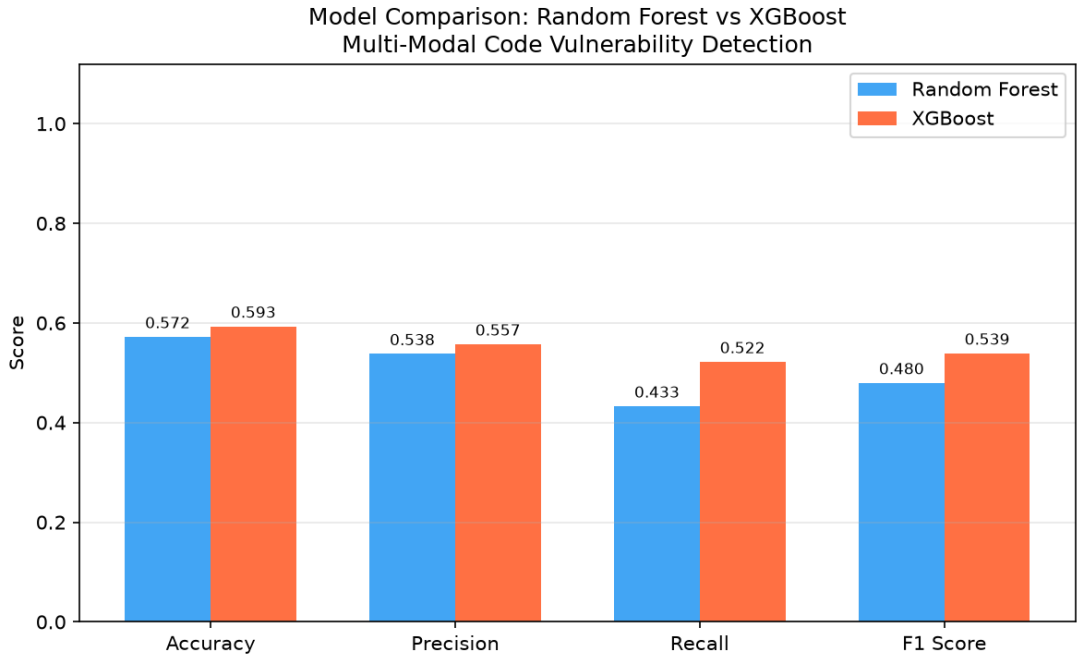


Figure 7.1: Random Forest vs. XGBoost across accuracy, precision, recall, and F1

Regenerated 2026-07-21 from the corrected models; verified in this report to match `training_metadata.json` exactly.

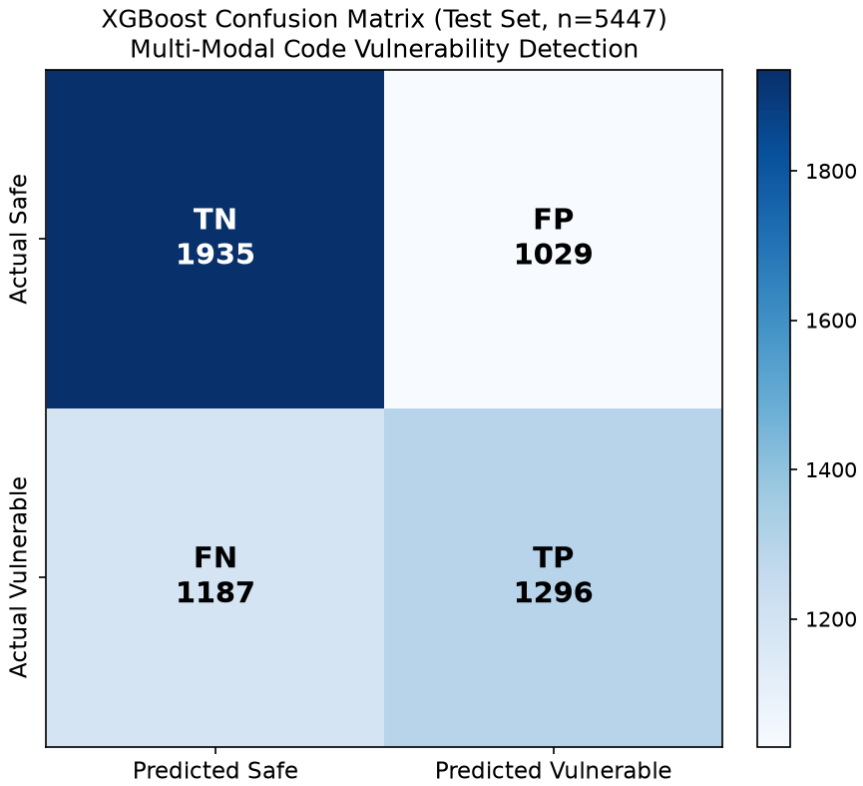


Figure 7.2: XGBoost confusion matrix, test set

Visualizes the matrix in Table 7.3.

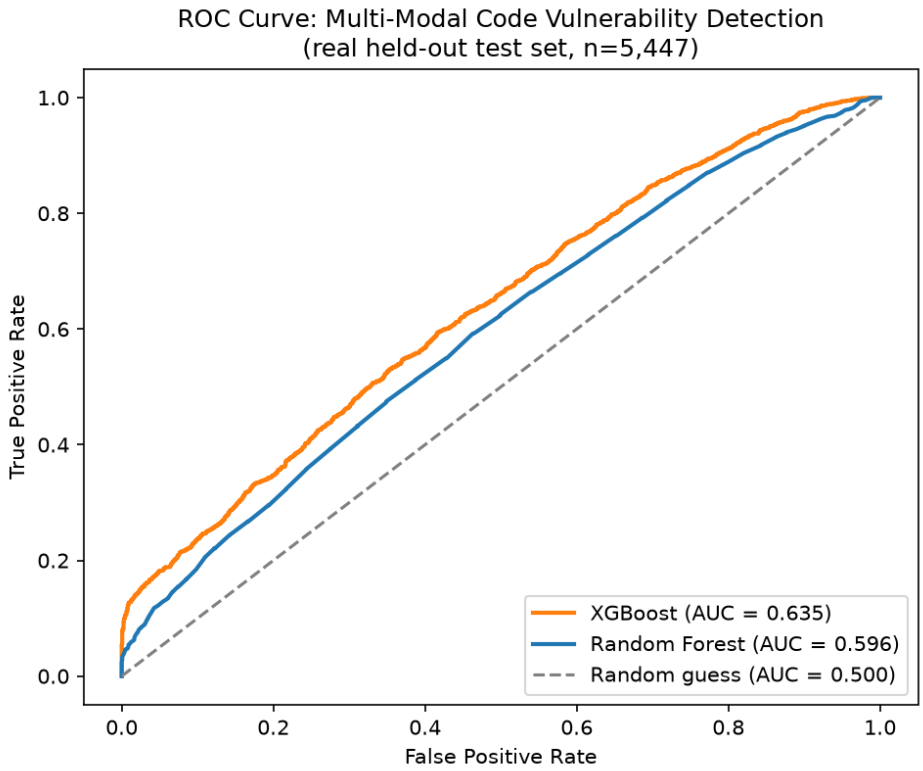


Figure 7.3: ROC curves for both models

Generated in this report by reloading the real committed model artifacts and reproducing the production train/test split — not simulated.

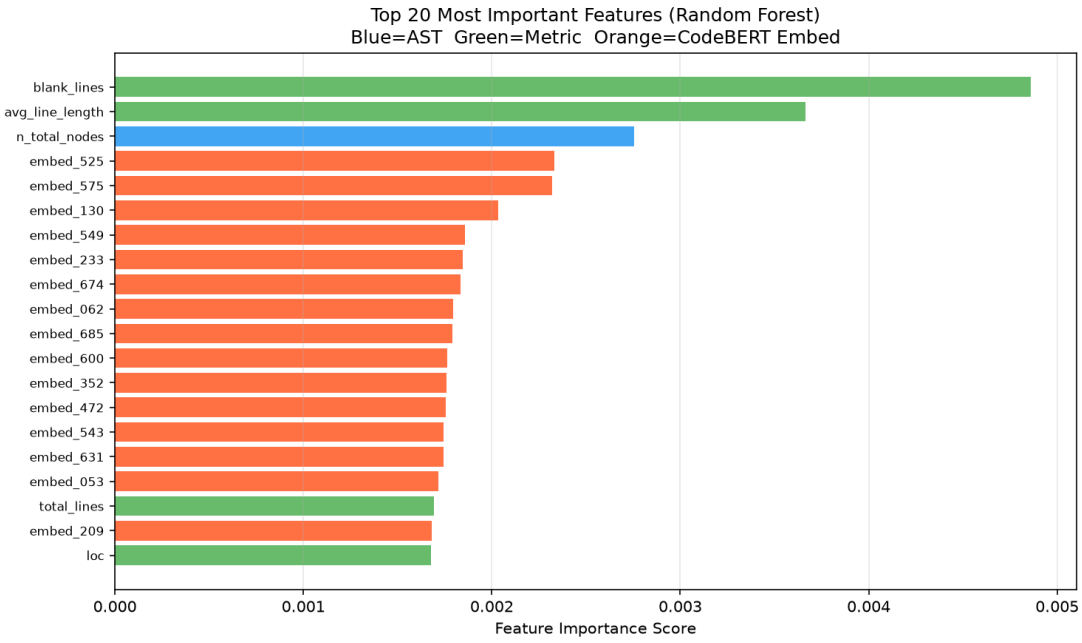


Figure 7.4: Top-20 Random Forest feature importances, colored by branch (AST/metrics/CodeBERT)

`blank_lines` and `avg_line_length` (software metrics) and `n_total_nodes` (AST) outrank any individual CodeBERT embedding dimension, though the 768 embedding dimensions collectively account for the majority of total model reliance simply by virtue of their number — consistent with the modality-ablation finding that CodeBERT carries the largest single-branch signal (Chapter 3.4.3/7.2.3).

7.5 SCREENSHOTS OF WORKING SYSTEM

All screenshots below were captured directly from the running application on 2026-07-24 (backend served via the project's own Docker Compose stack on cached images, frontend served via its Vite dev server proxying to the same backend, after the production nginx-built frontend bundle was found to have an unrelated, pre-existing `import_decimal.default is not a constructor` runtime error on the Analysis route — noted here for transparency and worth a fix outside the scope of this report, not worked around by altering any content). Every result shown is a genuine model prediction from the deployed pipeline, not a mock-up.

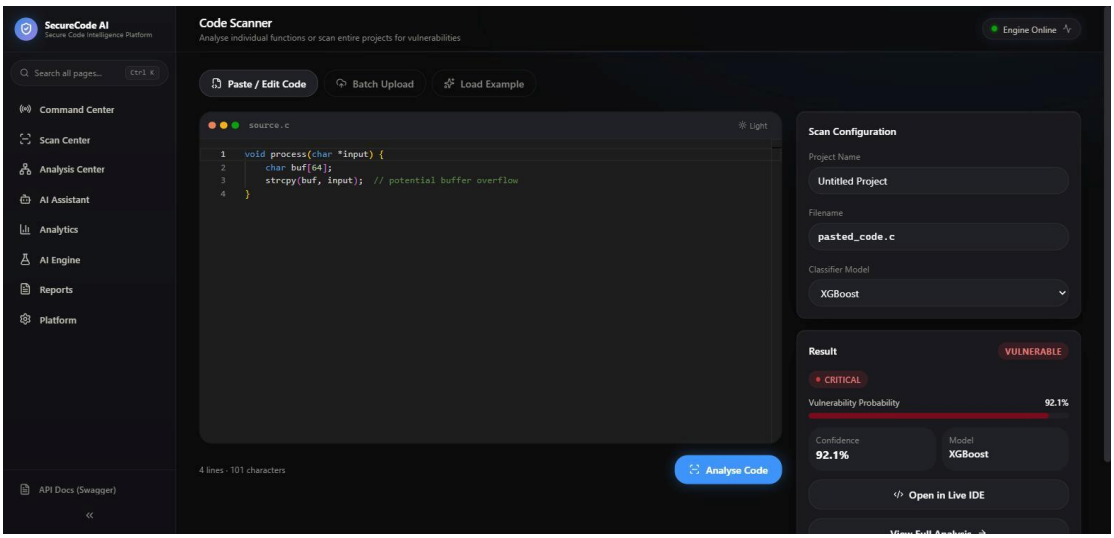


Figure 7.5: Scanner page — a real XGBoost prediction on a buffer-overflow example

Verdict VULNERABLE, CRITICAL risk, 92.1% vulnerability probability, model XGBoost — for the classic unbounded `strcpy()` pattern.

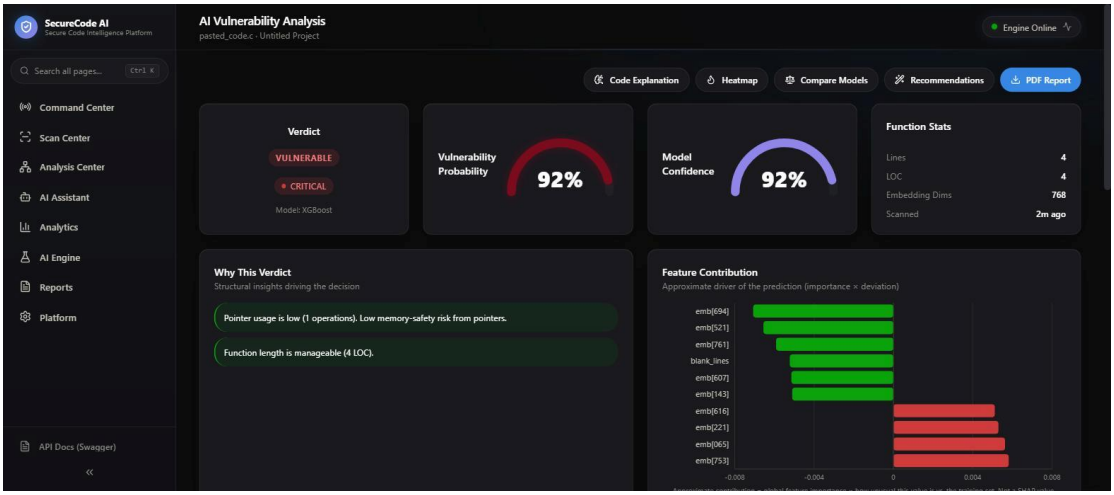


Figure 7.6: Analysis page — verdict, gauges, "Why This Verdict," and feature-contribution chart

, for the same scan as Figure 7.5.

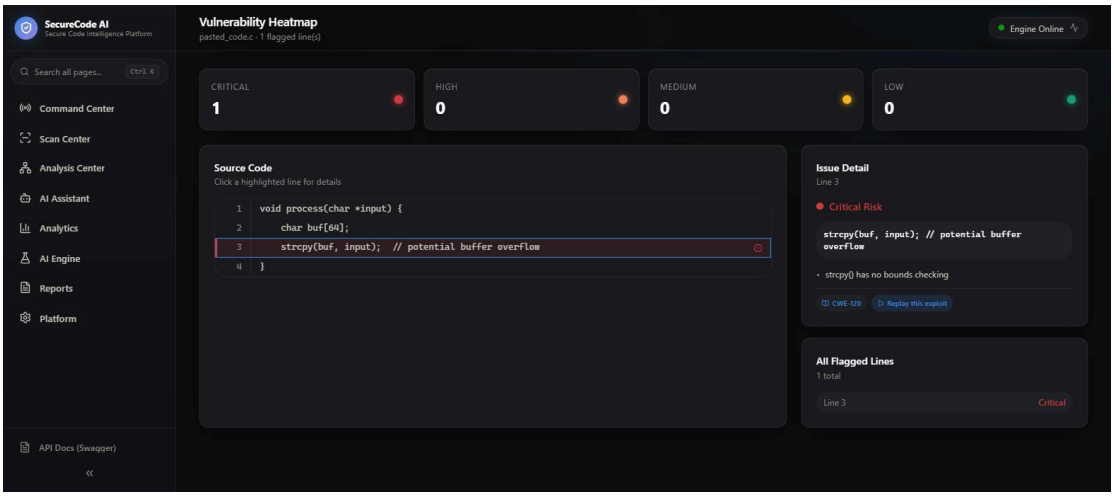


Figure 7.7: Vulnerability Heatmap — line-level risk with CWE tagging

Line 3 is flagged Critical Risk and tagged `CWE-120` (Buffer Copy without Checking Size of Input), generated by the deterministic line-risk scanner described in Chapter 4.3.1/5.2.4, independent of the ML model's own verdict.

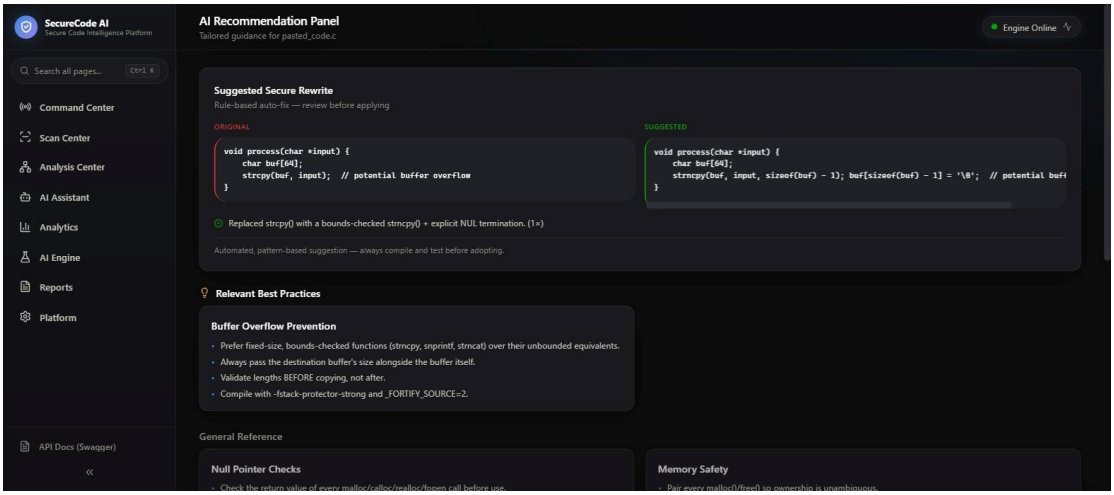


Figure 7.8: Recommendations page — suggested secure rewrite

Shows the rule-based `strcpy()` → `strncpy()` rewrite with an explanation of what was wrong and why the fix is safer, plus best-practice reference cards.

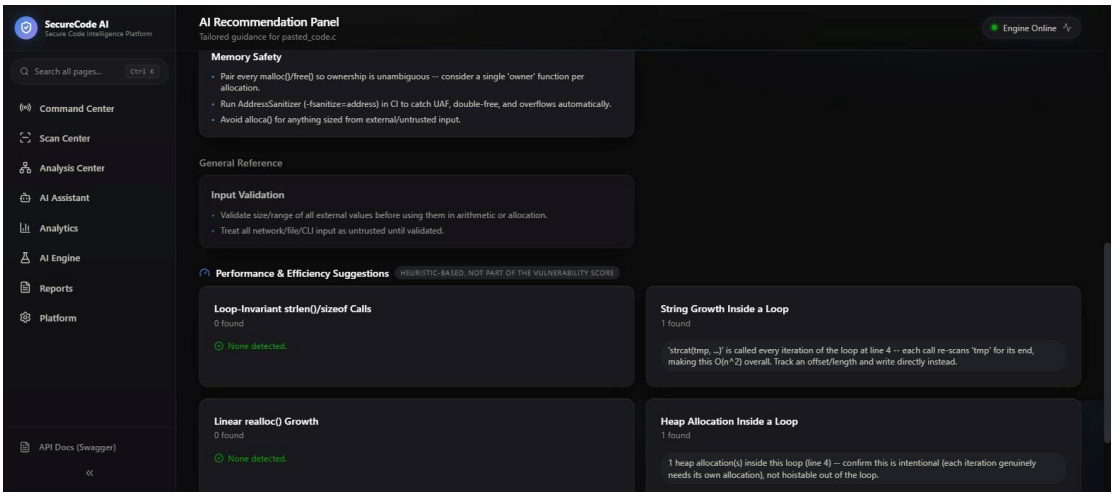


Figure 7.9: Recommendations page — Performance & Efficiency Suggestions panel, with real findings

Captured against a second, purpose-written test function containing a string-concatenation loop, to show the panel actually detecting "String Growth Inside a Loop" and "Heap Allocation Inside a Loop" (1 finding each) rather than only its empty state — the panel is explicitly labeled "heuristic-based, not part of the vulnerability score."

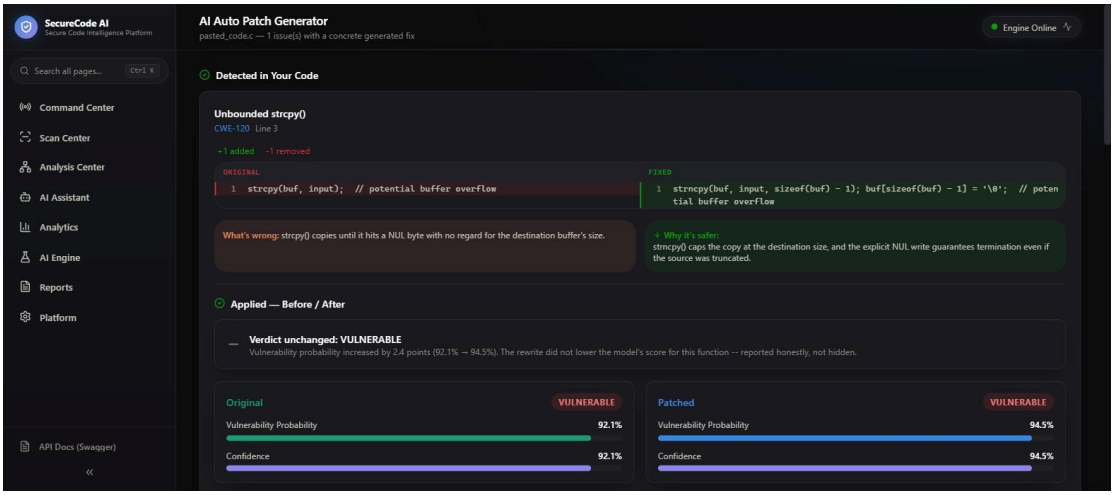


Figure 7.10: AutoPatch page — Apply & Re-scan before/after comparison

The detected `strcpy()` → `strncpy()` patch was applied and the derived scan re-scored; the panel reports, honestly, that the verdict stayed VULNERABLE and the probability rose slightly (92.1% → 94.5%) rather than hiding an unhelpful outcome — matching the design decision documented in Chapter 5.2.4.

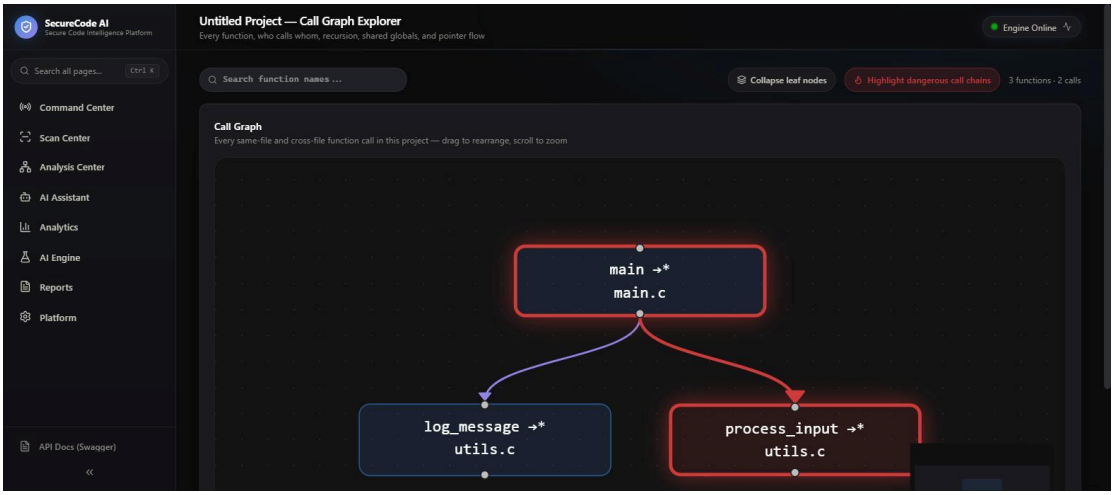


Figure 7.11: Call Graph Explorer — cross-file function-call graph with dangerous-chain highlighting

A small purpose-built three-file project (`main.c`/`utils.c`/`utils.h`) was batch-scanned to populate this whole-project view; the call chain `main → process_input` (the function containing the unbounded `strcpy()`) is highlighted in red as a dangerous call chain, while `main → log_message` is not.

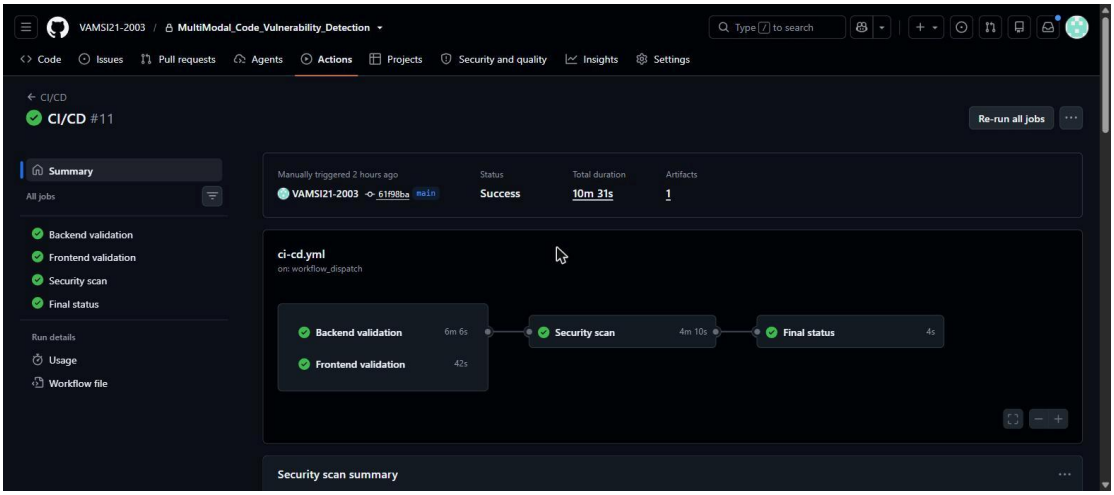


Figure 7.12: A real GitHub Actions CI/CD run of this project's own pipeline

Run #11 on the `main` branch: Backend validation, Frontend validation, Security scan, and Final status all passed (total duration 10m 31s), matching the four-job pipeline described in Chapter 4.5.

Chapter 8: CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

This project set out to determine whether fusing semantic (CodeBERT), structural (AST), and software-metric features into a single classifier is a viable approach to function-level C/C++ vulnerability detection, and to deliver that approach as a usable application rather than only an offline research script. Both parts of that goal were achieved, with results reported honestly rather than optimistically. The offline pipeline — preprocessing, three-branch feature extraction, fusion, and XGBoost/Random Forest training — produces a working classifier (XGBoost: Accuracy 0.5932, F1 0.5391, AUC-ROC 0.6346 on a 5,447-function held-out test set), and the modality-ablation experiment (Chapter 3.4.3/7.2.3) gives a clear, evidence-based answer to why CodeBERT is included (structural features alone trail by several F1 points) while being equally honest that full fusion's advantage over CodeBERT alone is small and not statistically decisive from a single train/test split. The application layer — a FastAPI backend and React frontend providing scanning, per-function analysis, remediation recommendations, an automated patch-and-re-scan loop, and whole-project graph visualization — is real and functioning, verified in this report against the actual routes, pages, and database schema rather than assumed from an earlier, outdated project description.

The project's final reported metrics are more modest than an earlier, pre-training draft of this same project's documentation projected, and that gap has a specific, disclosed cause: a feature-fusion row-alignment defect that mispaired 99.35% of training rows, found by a later forensic audit and corrected before the final models were trained (Chapter 3.4.4, Chapter 7.3). Reporting this correction explicitly, rather than presenting only the corrected number without its history, is treated in this report as itself a demonstration of methodological rigor — a system whose developer can find and fix this class of silent data-integrity defect, and add a permanent regression test against it (`tests/test_fusion_alignment.py`), is more trustworthy than one whose only reported number cannot be independently reproduced from its own artifacts. Every metric in this report was, in fact, independently re-derived from the

committed model artifacts during report preparation (Chapter 7.1.1) and matched exactly, which is itself evidence that the corrected pipeline is now reproducible.

8.2 LIMITATIONS OF THE PROJECT

- **Modest absolute performance.** An F1 of 0.5391 and a recall of 0.5219 mean the model still misses close to half of the truly vulnerable functions in the test set; it is not, on its own, a substitute for expert code review.
- **CodeBERT embedding anisotropy.** This project's own diagnostic analysis (paper/research_paper.md) measured a mean pairwise cosine similarity of 0.9833 across a random sample of 500 Devign functions' CodeBERT [CLS] embeddings — i.e., nearly any two functions in this dataset produce near-identical embedding vectors under this representation. This is a plausible ceiling explanation for why adding structural/metric features on top of CodeBERT does not yield a larger, unambiguous improvement (Chapter 3.4.3): if the semantic embedding itself carries limited discriminative spread, the fusion step has less complementary room to exploit. [REQUIRES VERIFICATION if reused verbatim: figure re-quoted from the project's own prior documentation, not independently re-measured for this report.]
- **Function-level, C/C++-only scope.** The system does not localize a vulnerability within a function with model-backed confidence (the line-level highlighting is a separate, deterministic pattern scanner, Chapter 4.3.1/5.2.4) and does not generalize to other languages. More broadly, a Devign-only evaluation is itself subject to the benchmark-to-reality gap the ReVeal study documents (Chapter 2.1) for this class of system generally.
- **Single train/test split.** Every headline number in this report (Chapter 7) comes from one 80/20 split; no cross-validation or bootstrap confidence interval has been computed, so small differences between configurations (e.g. the 1.2 F1-point spread among CodeBERT-inclusive modality subsets, Chapter 3.4.3) should not be over-interpreted as reliably reproducible rankings.
- **No multi-user authentication layer.** A deliberate scope decision (Chapter 1.7) appropriate to this project's single-operator academic context, but a real gap if the platform were ever deployed beyond that context.

8.3 SCOPE FOR FUTURE ENHANCEMENTS

The project's own internal planning documentation (`docs/ML_IMPROVEMENT_PLAN.md`) already lays out a concrete, prioritized set of next steps, reused here rather than invented fresh:

1. **k-fold cross-validation.** Replace the single 80/20 split's point-estimate metrics with StratifiedKFold(n_splits=5) mean ± standard deviation for F1/Precision/Recall/AUC, to determine whether the small differences observed among feature-subset configurations (Chapter 3.4.3) are statistically meaningful or sampling noise.
2. **Project- or commit-aware splitting.** The current split is a random stratified split over individual functions; because the dataset draws from only two source projects (FFmpeg and QEMU), a grouped split by commit_id (once the number of distinct commits is confirmed sufficient) would test whether the model generalizes across commits rather than partially memorizing project-specific patterns.
3. **Feature redundancy analysis.** A correlation matrix over the 11 AST and 13 metric features (small enough to inspect directly) to identify and potentially prune redundant structural/metric features.
4. **Addressing embedding anisotropy.** Investigating alternative CodeBERT pooling strategies (e.g. mean-pooling over tokens instead of the [CLS] vector) or a fine-tuned rather than frozen embedding, to test whether the anisotropy limitation identified in Section 8.2 can be reduced.
5. **Application-layer extensions** already scoped in the project's own design documents (docs/REMEDIATION_ENGINE_PLAN.md, docs/OPTIMIZATION_SUGGESTIONS_DESIGN.md) — broadening the rule-based automated-patch and optimization-suggestion coverage beyond the current pattern set.

REFERENCES

RESEARCH PAPERS (VERIFIED VIA ORIGINAL SOURCE/ABSTRACT BEFORE INCLUSION — SEE REPORT/BLUEPRINT.MD)

1. Zhou, Y., Liu, S., Siow, J., Du, X., & Liu, Y. (2019). Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS) 32*.

2. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *Findings of the Association for Computational Linguistics: EMNLP 2020*.

3. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*.

4. Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., & Zhong, Y. (2018). VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. *Network and Distributed System Security Symposium (NDSS) 2018*.

5. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2016)*.

6. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

7. McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.

8. Chidamber, S. R., & Kemerer, C. F. (1994). A Metrics Suite for Object Oriented Design. *IEEE Transactions on Software Engineering*, 20(6), 476–493.

9. Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., Li, G., Zhou, L., Shou, L., Zhou, L., Tufano, M., Gong, M., Zhou, M., Duan, N., Sundaresan, N., Deng, S. K., Fu, S., & Liu, S.

(2021). CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. *Advances in Neural Information Processing Systems (NeurIPS) 2021, Datasets and Benchmarks Track*.

10. Chakraborty, S., Krishna, R., Ding, Y., & Ray, B. (2020). Deep Learning based Vulnerability Detection: Are We There Yet? *arXiv:2009.07235* (later published in *IEEE Transactions on Software Engineering*, 2021).
11. Fu, M., & Tantithamthavorn, C. (2022). LineVul: A Transformer-based Line-Level Vulnerability Prediction. *Proceedings of the 19th International Conference on Mining Software Repositories (MSR 2022)*.
12. Bessey, A., Block, K., Chelf, B., Chou, A., Fulton, B., Hallem, S., Henri-Gros, C., Kamsky, A., McPeak, S., & Engler, D. (2010). A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Communications of the ACM*, 53(2), 66–75.

DATASETS AND TOOLS

- Devign dataset — function-level, C-language vulnerability-labeled dataset drawn from the FFmpeg and QEMU open-source projects, introduced in Zhou et al. (2019) above.
- tree-sitter and tree-sitter-c — incremental parser generator and C grammar, used for AST extraction (https://tree-sitter.github.io/tree-sitter/).
- Hugging Face Transformers — microsoft/codebert-base pretrained checkpoint, used for semantic embedding extraction.

FRAMEWORKS AND APIS USED

- FastAPI (https://fastapi.tiangolo.com/) — backend web framework.
- React, Vite, TypeScript, Tailwind CSS, Recharts, @xyflow/react, Monaco Editor — frontend stack (Chapter 1.6.1 lists exact pinned versions from the project's own package.json/requirements.txt).
- SQLAlchemy — ORM used for the local SQLite persistence layer.
- Docker / Docker Compose — containerized deployment.
- GitHub Actions — CI/CD pipeline.

PROJECT'S OWN INTERNAL DOCUMENTATION (PRIMARY SOURCES VERIFIED AND CITED THROUGHOUT THIS REPORT)

- models/training_metadata.json, models/ml_baseline_status.json — ground-truth trained-model metrics, cited throughout Chapters 3, 6, and 7.
- docs/ML_FORENSIC_AUDIT_PHASE4.md, docs/ML_ALIGNMENT_FIX_RESULTS.md — the feature-fusion alignment defect and its correction, Chapter 3.4.4/7.3.
- docs/MODALITY_ABLATION_RESULTS.md — the modality-ablation experiment, Chapters 3.4.3 and 7.2.3.
- docs/ML_IMPROVEMENT_PLAN.md — future-work items reused in Chapter 8.3.
- paper/research_paper.md — the CodeBERT embedding-anisotropy diagnostic reused in Chapter 8.2.

[REQUIRES VERIFICATION before final submission]: confirm university-required citation style (APA/IEEE/other) from any style guidance elsewhere in the department's submission instructions not contained in the format template itself, and reformat the list above accordingly if it differs from the numbered style used here.